

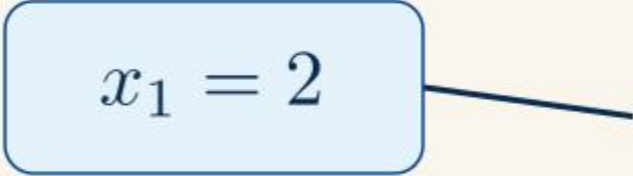
A Neural Unit

A weighted sum is followed by a nonlinear activation.

A Neural Unit

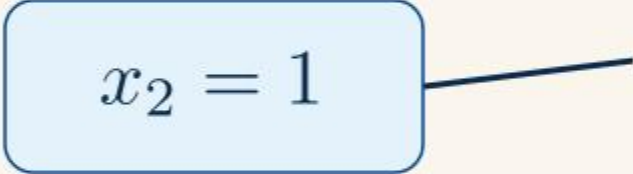
A weighted sum is followed by a nonlinear activation.

INPUTS



A light blue rounded rectangle containing the equation $x_1 = 2$. A black line extends from the right side of the rectangle.

$$x_1 = 2$$



A light blue rounded rectangle containing the equation $x_2 = 1$. A black line extends from the right side of the rectangle.

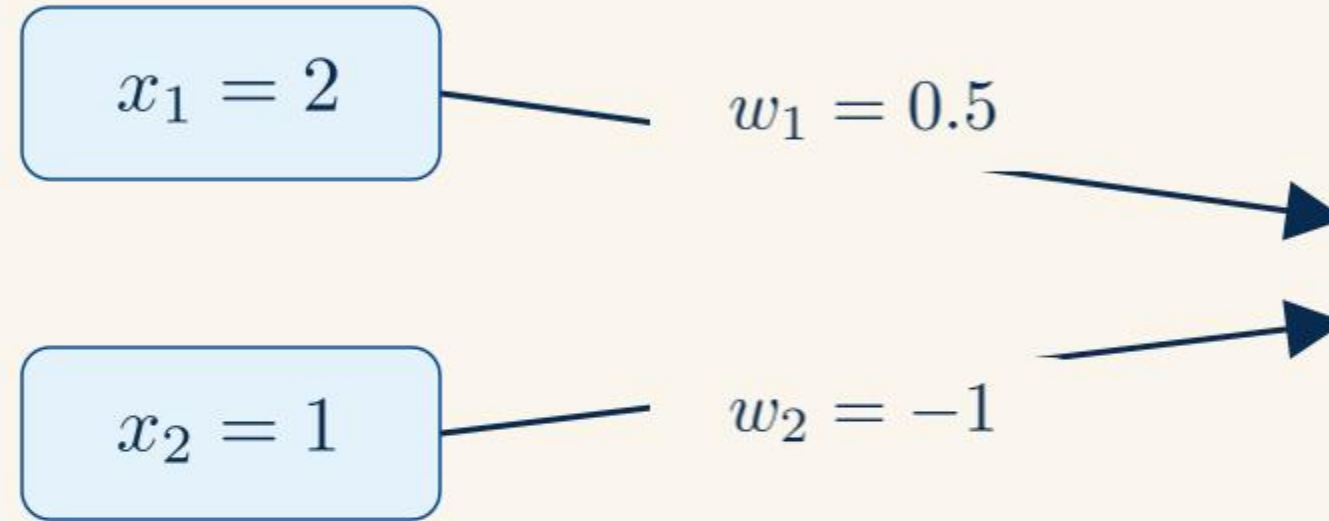
$$x_2 = 1$$

A Neural Unit

A weighted sum is followed by a nonlinear activation.

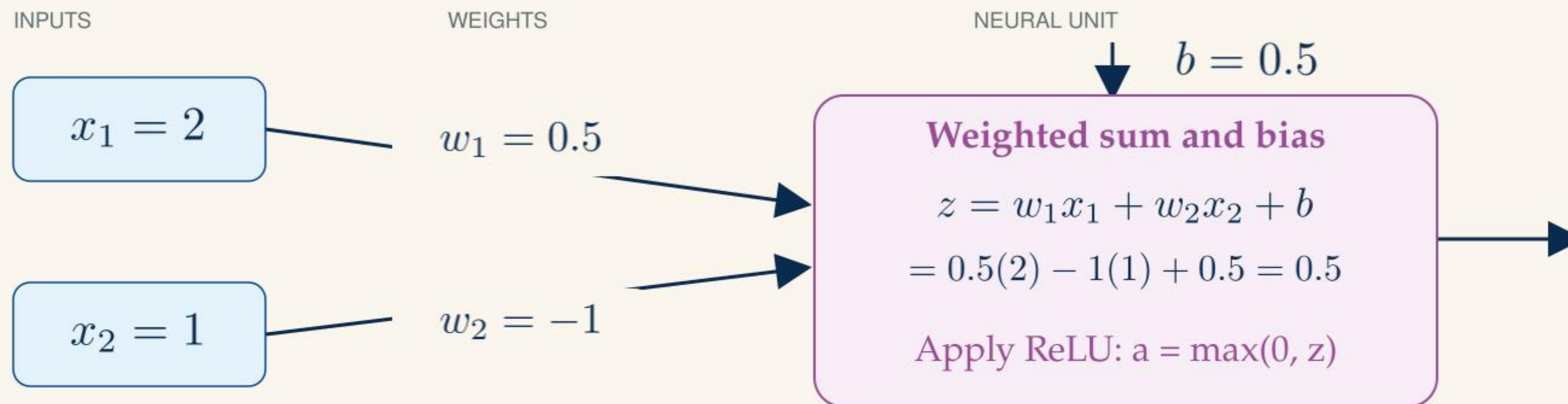
INPUTS

WEIGHTS



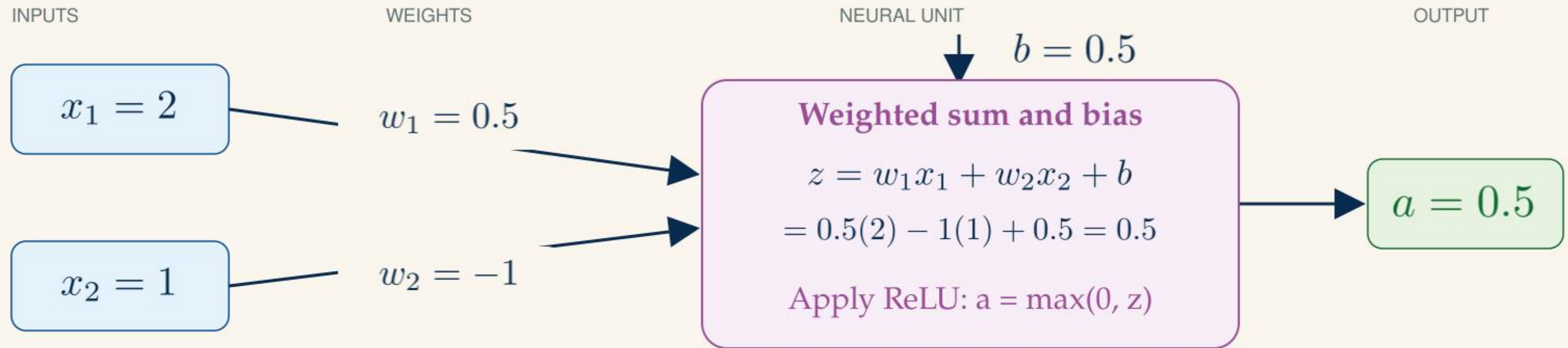
A Neural Unit

A weighted sum is followed by a nonlinear activation.



A Neural Unit

A weighted sum is followed by a nonlinear activation.



$$z = \mathbf{w}^T \mathbf{x} + b$$

$$a = g(z)$$

Course convention: z is the affine input; a names the unit activation.

The symbols y and $\hat{y}$ are reserved for observed targets and model predictions.

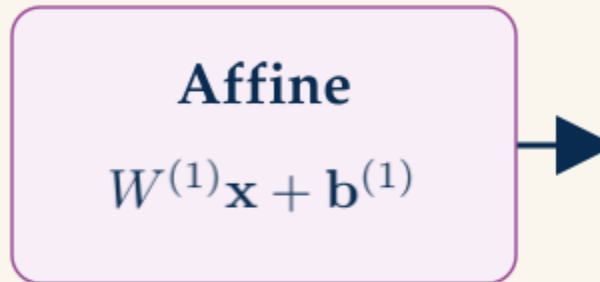
Linear Combinations and Nonlinear Layers

Each layer transforms the representation passed to the next.

Linear Combinations and Nonlinear Layers

Each layer transforms the representation passed to the next.

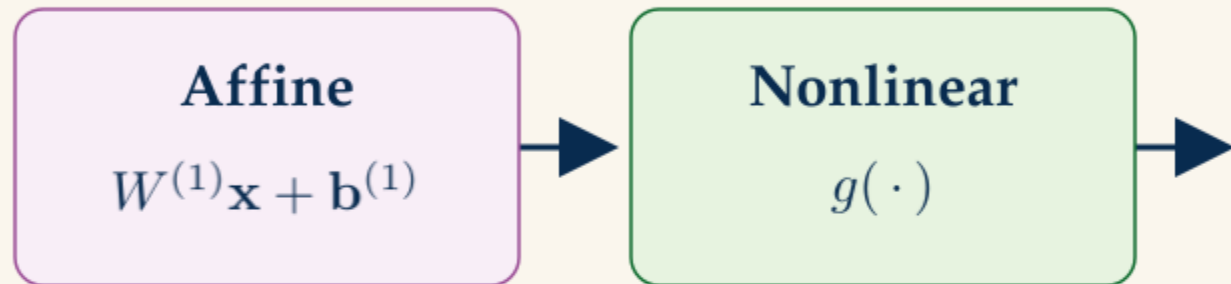
A NETWORK REPEATS TWO OPERATIONS



Linear Combinations and Nonlinear Layers

Each layer transforms the representation passed to the next.

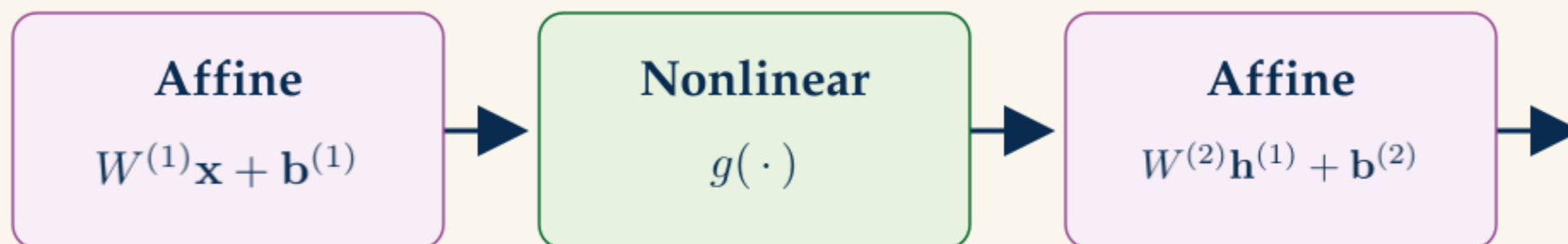
A NETWORK REPEATS TWO OPERATIONS



Linear Combinations and Nonlinear Layers

Each layer transforms the representation passed to the next.

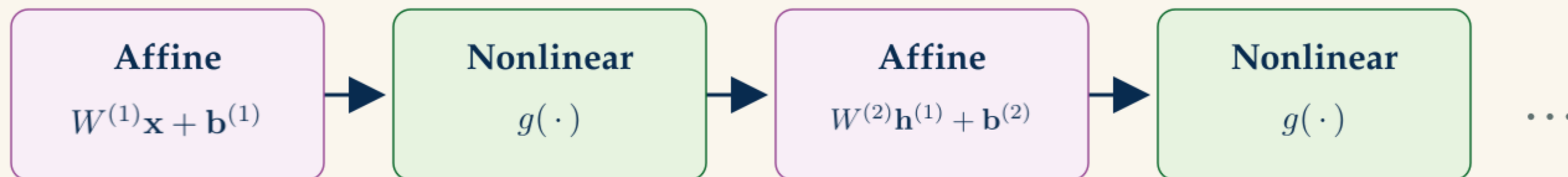
A NETWORK REPEATS TWO OPERATIONS



Linear Combinations and Nonlinear Layers

Each layer transforms the representation passed to the next.

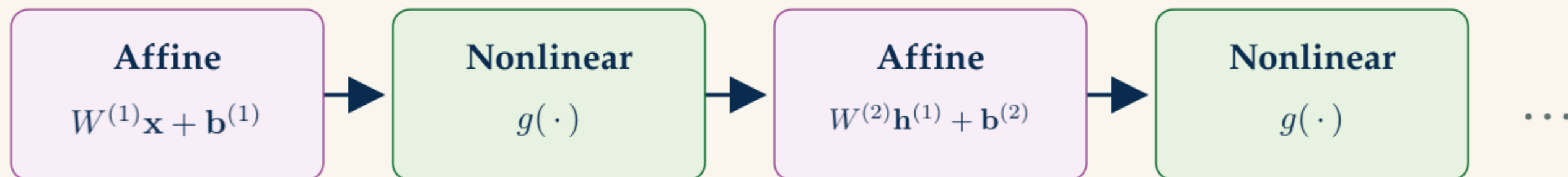
A NETWORK REPEATS TWO OPERATIONS



Linear Combinations and Nonlinear Layers

Each layer transforms the representation passed to the next.

A NETWORK REPEATS TWO OPERATIONS



ONE HIDDEN LAYER WITH A CLASSIFICATION OUTPUT

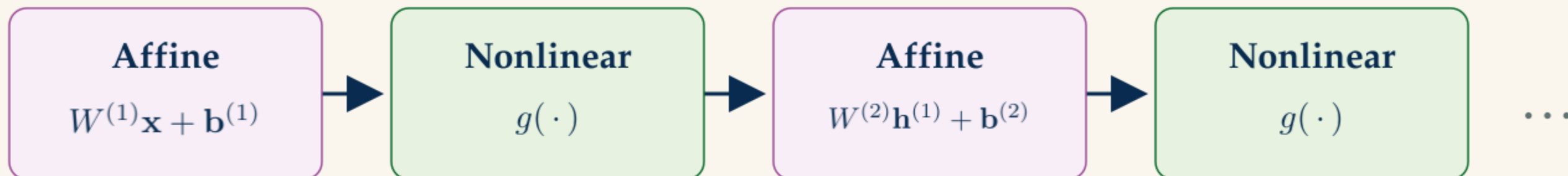
$$\mathbf{h} = \sigma(W\mathbf{x} + \mathbf{b})$$

Hidden representation

Linear Combinations and Nonlinear Layers

Each layer transforms the representation passed to the next.

A NETWORK REPEATS TWO OPERATIONS



ONE HIDDEN LAYER WITH A CLASSIFICATION OUTPUT

$$\mathbf{h} = \sigma(W\mathbf{x} + \mathbf{b})$$

Hidden representation

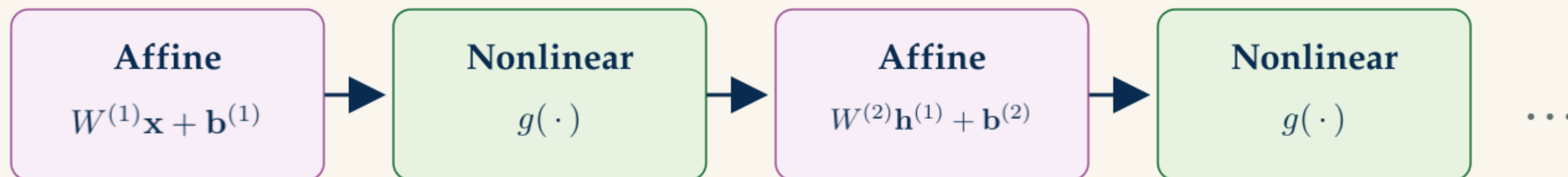
$$\mathbf{z} = U\mathbf{h}$$

Vocabulary or class logits

Linear Combinations and Nonlinear Layers

Each layer transforms the representation passed to the next.

A NETWORK REPEATS TWO OPERATIONS



ONE HIDDEN LAYER WITH A CLASSIFICATION OUTPUT

$$\mathbf{h} = \sigma(W\mathbf{x} + \mathbf{b})$$

Hidden representation

$$\mathbf{z} = U\mathbf{h}$$

Vocabulary or class logits

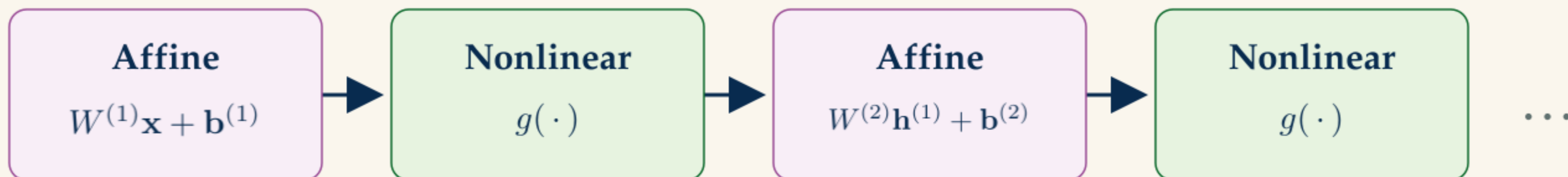
$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{z})$$

Predicted probabilities

Linear Combinations and Nonlinear Layers

Each layer transforms the representation passed to the next.

A NETWORK REPEATS TWO OPERATIONS



ONE HIDDEN LAYER WITH A CLASSIFICATION OUTPUT

$$\mathbf{h} = \sigma(W\mathbf{x} + \mathbf{b})$$

Hidden representation

$$\mathbf{z} = U\mathbf{h}$$

Vocabulary or class logits

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{z})$$

Predicted probabilities

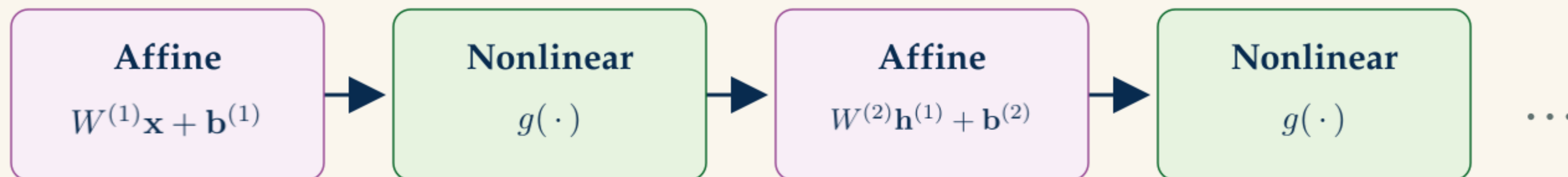
The activation is applied element by element; softmax normalizes the output scores.

Notation used from here: $\hat{y}$ is the model prediction; y is the observed target.

Linear Combinations and Nonlinear Layers

Each layer transforms the representation passed to the next.

A NETWORK REPEATS TWO OPERATIONS



ONE HIDDEN LAYER WITH A CLASSIFICATION OUTPUT

$$\mathbf{h} = \sigma(W\mathbf{x} + \mathbf{b})$$

Hidden representation

$$\mathbf{z} = U\mathbf{h}$$

Vocabulary or class logits

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{z})$$

Predicted probabilities

The activation is applied element by element; softmax normalizes the output scores.

Notation used from here: $\hat{y}$ is the model prediction; y is the observed target.

Why nonlinearities matter Without them, stacked affine layers reduce to one affine transformation.

Learning the Network Parameters

Backpropagation computes gradients; the optimizer updates parameters.

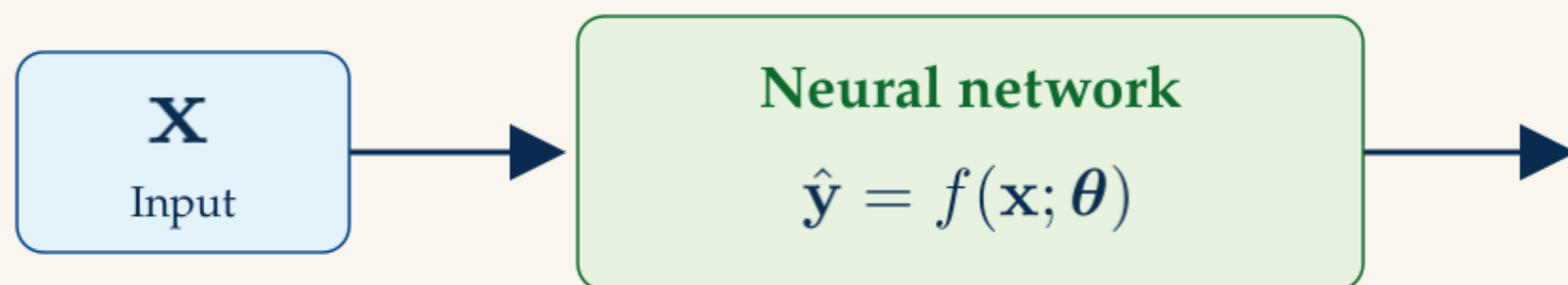
FORWARD PASS



Learning the Network Parameters

Backpropagation computes gradients; the optimizer updates parameters.

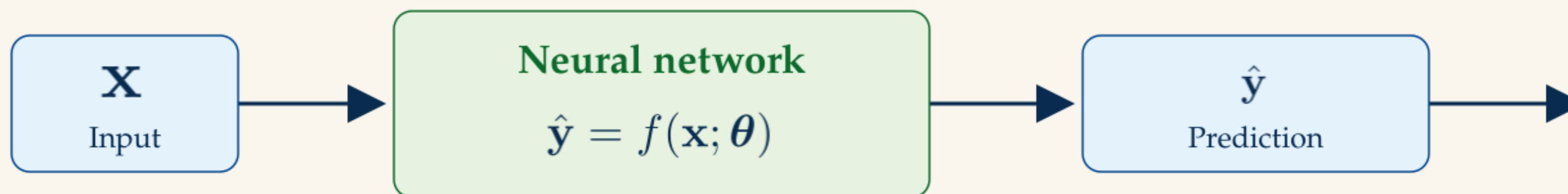
FORWARD PASS



Learning the Network Parameters

Backpropagation computes gradients; the optimizer updates parameters.

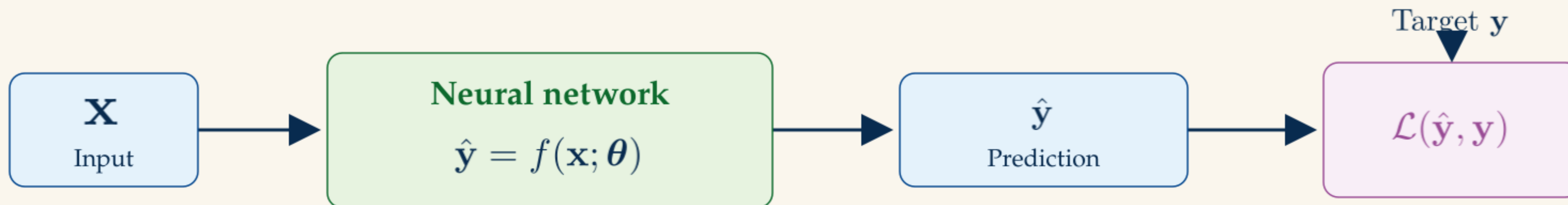
FORWARD PASS



Learning the Network Parameters

Backpropagation computes gradients; the optimizer updates parameters.

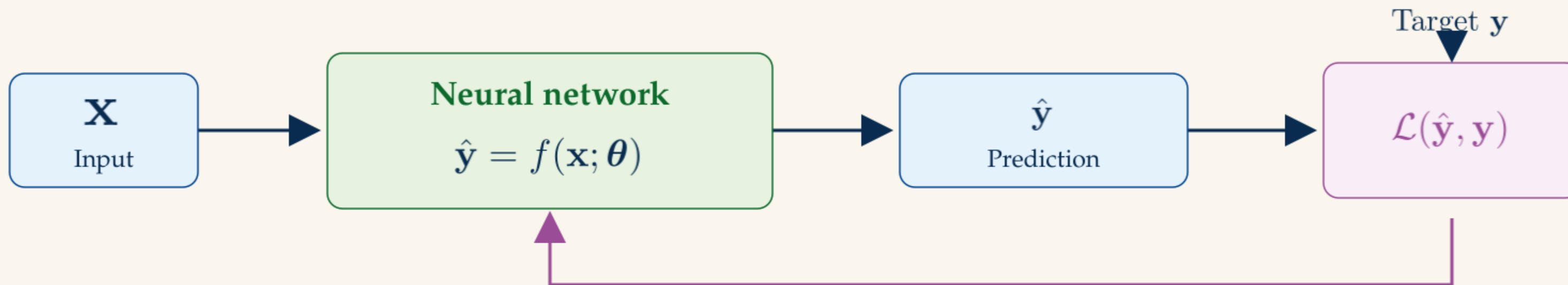
FORWARD PASS



Learning the Network Parameters

Backpropagation computes gradients; the optimizer updates parameters.

FORWARD PASS



BACKPROPAGATION

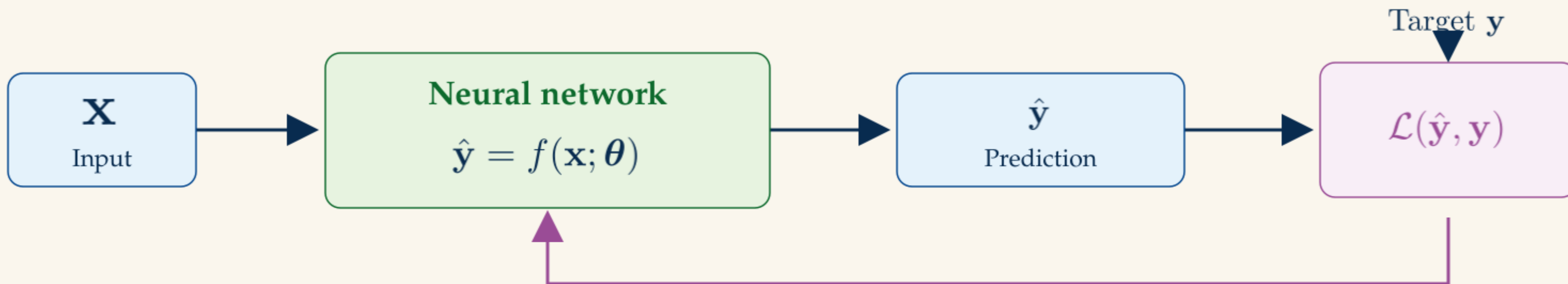
Compute the loss gradient with respect to every trainable parameter.

$$\nabla_{\boldsymbol{\theta}} \mathcal{L}$$

Learning the Network Parameters

Backpropagation computes gradients; the optimizer updates parameters.

FORWARD PASS



BACKPROPAGATION

Compute the loss gradient with respect to every trainable parameter.

$$\nabla_{\boldsymbol{\theta}} \mathcal{L}$$

OPTIMIZER UPDATE

$$\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} - \eta \nabla_{\boldsymbol{\theta}} \mathcal{L}$$

$$\boldsymbol{\theta} = \{W^{(i)}, \mathbf{b}^{(i)}\}_{i=1}^L$$

$\boldsymbol{\theta}$ collects all trainable weights and biases; η is the learning rate.

Backpropagation supplies the gradient; the optimizer performs the update.

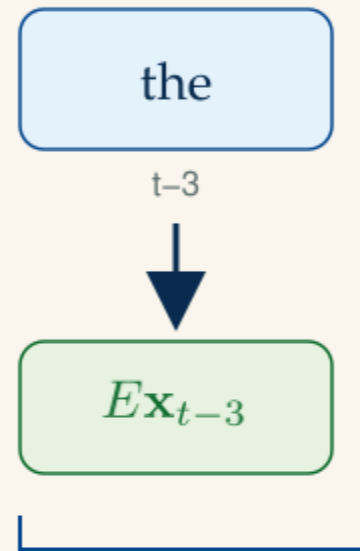
A Feedforward Neural Language Model

A fixed window of embeddings predicts one vocabulary distribution.

A Feedforward Neural Language Model

A fixed window of embeddings predicts one vocabulary distribution.

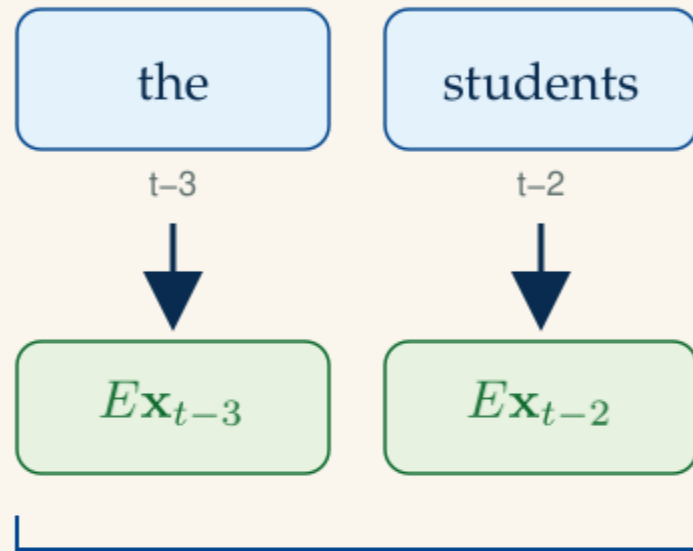
FIXED CONTEXT WINDOW



A Feedforward Neural Language Model

A fixed window of embeddings predicts one vocabulary distribution.

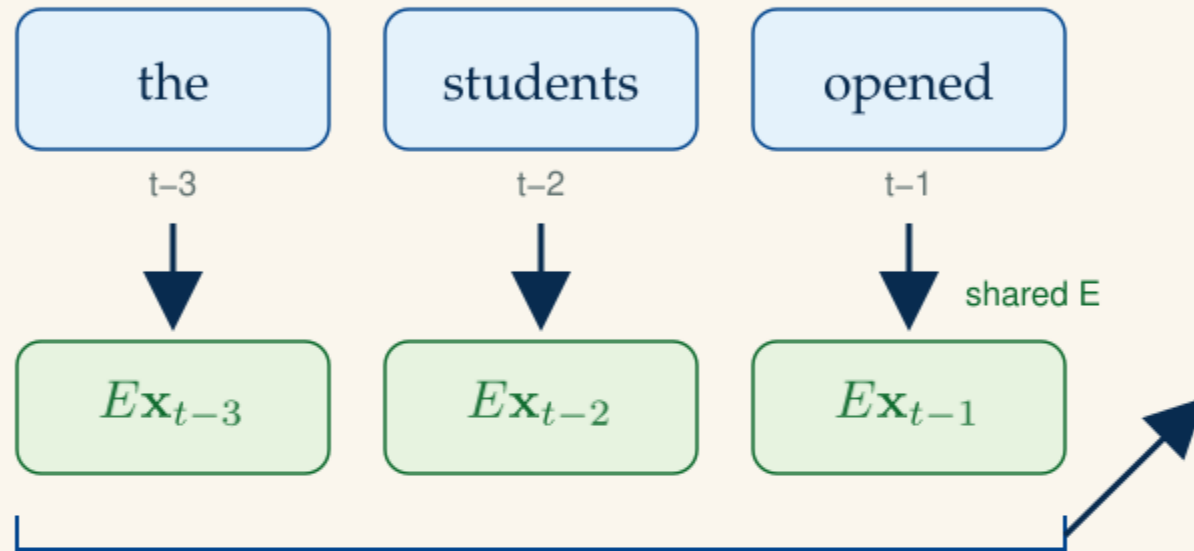
FIXED CONTEXT WINDOW



A Feedforward Neural Language Model

A fixed window of embeddings predicts one vocabulary distribution.

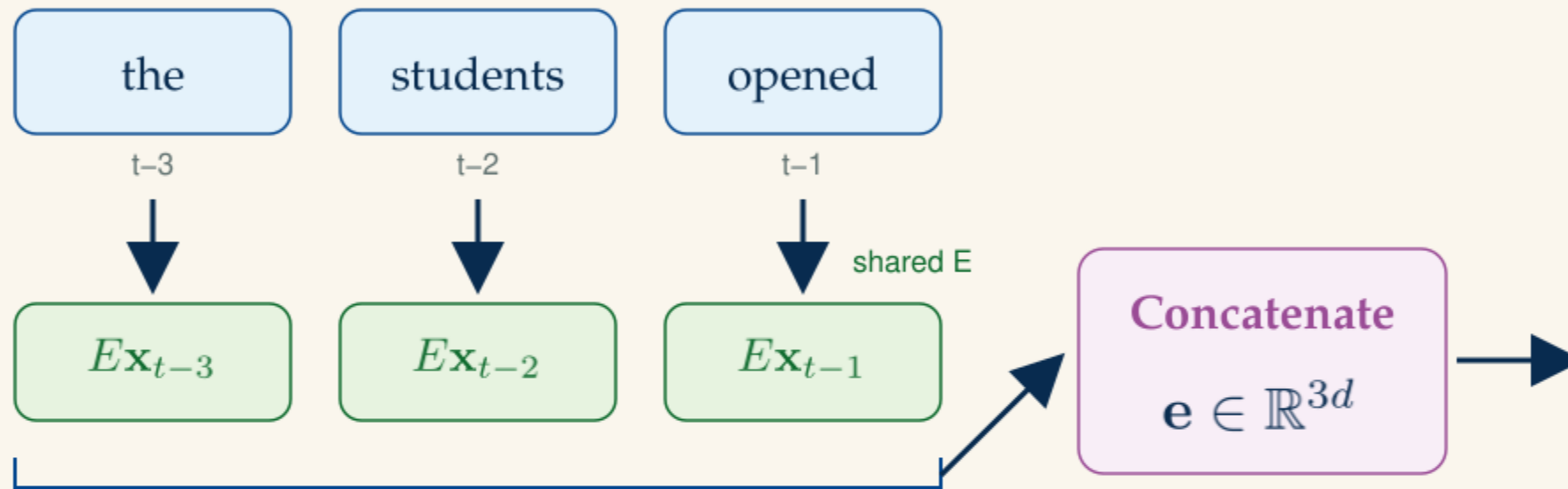
FIXED CONTEXT WINDOW



A Feedforward Neural Language Model

A fixed window of embeddings predicts one vocabulary distribution.

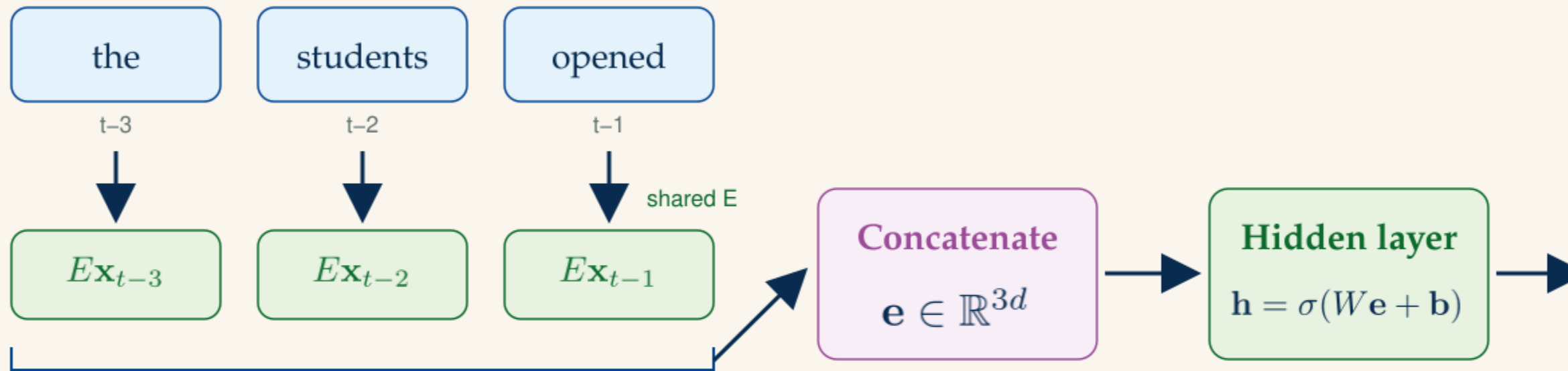
FIXED CONTEXT WINDOW



A Feedforward Neural Language Model

A fixed window of embeddings predicts one vocabulary distribution.

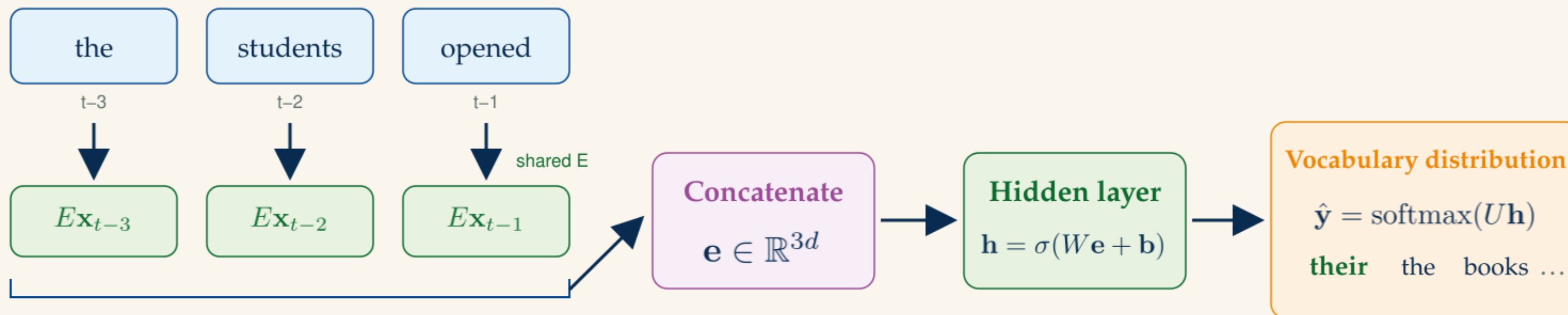
FIXED CONTEXT WINDOW



A Feedforward Neural Language Model

A fixed window of embeddings predicts one vocabulary distribution.

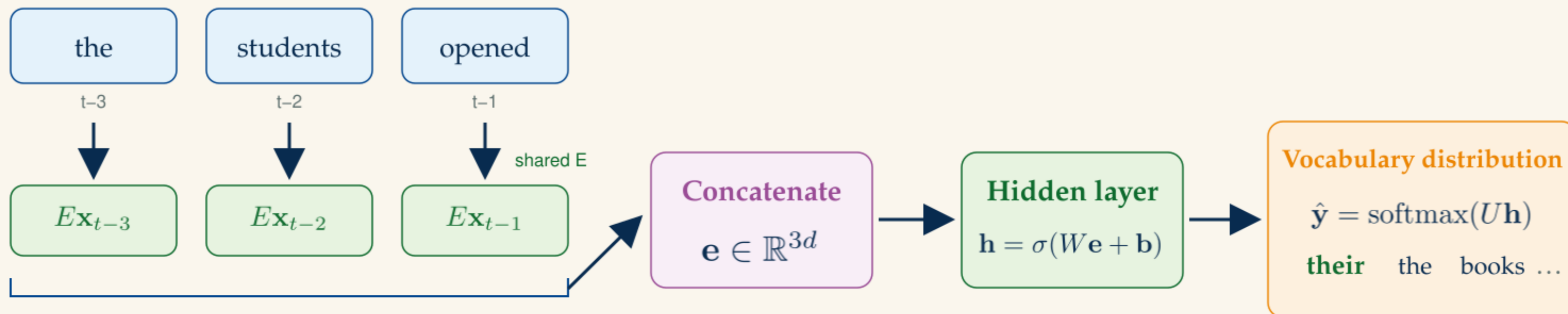
FIXED CONTEXT WINDOW



A Feedforward Neural Language Model

A fixed window of embeddings predicts one vocabulary distribution.

FIXED CONTEXT WINDOW



$$\mathbf{e} = [E\mathbf{x}_{t-3}; E\mathbf{x}_{t-2}; E\mathbf{x}_{t-1}]$$

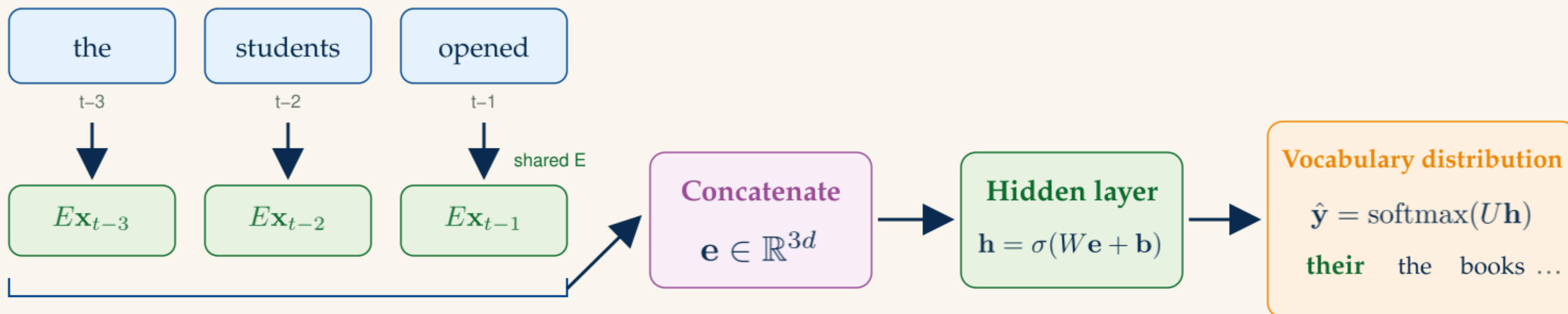
$$\mathbf{z} = U\mathbf{h}$$

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{z})$$

A Feedforward Neural Language Model

A fixed window of embeddings predicts one vocabulary distribution.

FIXED CONTEXT WINDOW



$$\mathbf{e} = [E\mathbf{x}_{t-3}; E\mathbf{x}_{t-2}; E\mathbf{x}_{t-1}]$$

$$\mathbf{z} = U\mathbf{h}$$

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{z})$$

The output has one probability for every token in the vocabulary.

The context has order, but its length is fixed by the concatenated input.

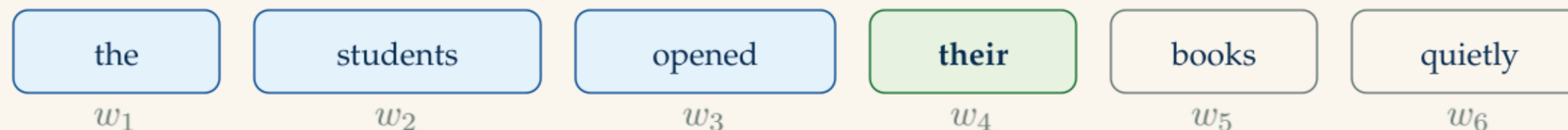
Each Position Supplies a Training Example

Ordinary text supplies both the context and the observed next-token target.

Each Position Supplies a Training Example

Ordinary text supplies both the context and the observed next-token target.

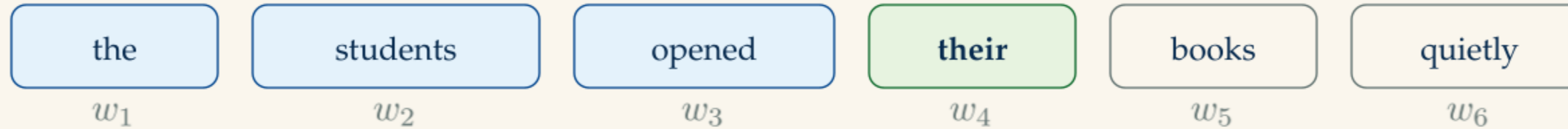
ONE SENTENCE PRODUCES SUCCESSIVE CONTEXT–TARGET PAIRS



Each Position Supplies a Training Example

Ordinary text supplies both the context and the observed next-token target.

ONE SENTENCE PRODUCES SUCCESSIVE CONTEXT–TARGET PAIRS



At position t

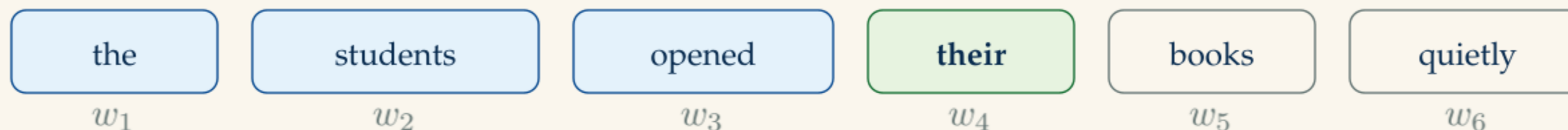
$$\mathcal{L}_t = -\log \hat{\mathbf{y}}_t[w_t]$$

Negative log probability assigned to the observed next token.

Each Position Supplies a Training Example

Ordinary text supplies both the context and the observed next-token target.

ONE SENTENCE PRODUCES SUCCESSIVE CONTEXT-TARGET PAIRS



At position t

$$\mathcal{L}_t = -\log \hat{\mathbf{y}}_t[w_t]$$

Negative log probability assigned to the observed next token.

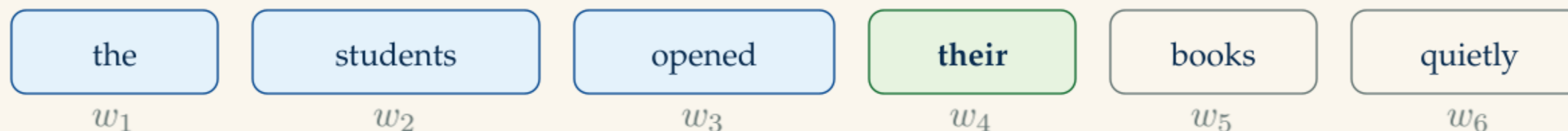
TRAINING EXAMPLE 1



Each Position Supplies a Training Example

Ordinary text supplies both the context and the observed next-token target.

ONE SENTENCE PRODUCES SUCCESSIVE CONTEXT-TARGET PAIRS

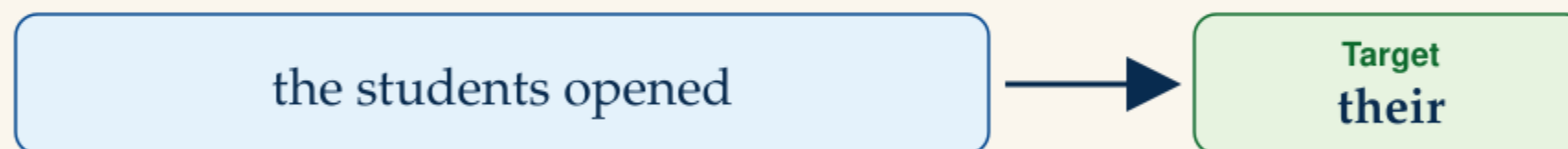


At position t

$$\mathcal{L}_t = -\log \hat{y}_t[w_t]$$

Negative log probability assigned to the observed next token.

TRAINING EXAMPLE 1



$$\mathcal{L}_4 = -\log \hat{y}_4[\text{their}]$$

TRAINING EXAMPLE 2

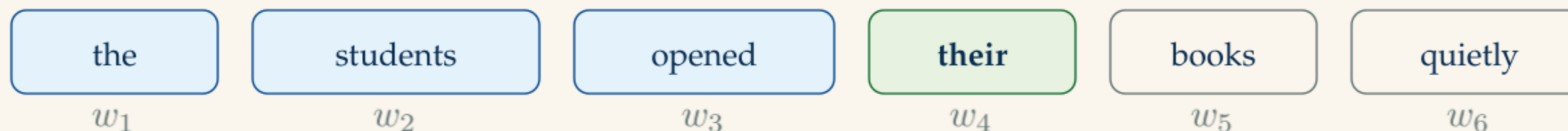


$$\mathcal{L}_5 = -\log \hat{y}_5[\text{books}]$$

Each Position Supplies a Training Example

Ordinary text supplies both the context and the observed next-token target.

ONE SENTENCE PRODUCES SUCCESSIVE CONTEXT-TARGET PAIRS



At position t

$$\mathcal{L}_t = -\log \hat{y}_t[w_t]$$

Negative log probability assigned to the observed next token.

TRAINING EXAMPLE 1

the students opened



Target
their

$$\mathcal{L}_4 = -\log \hat{y}_4[\text{their}]$$

TRAINING EXAMPLE 2

students opened their



Target
books

$$\mathcal{L}_5 = -\log \hat{y}_5[\text{books}]$$

Training Corpus targets become mini-batched examples.

Inference Select or sample; shift the window; repeat.

Joint update Backpropagation updates the embedding matrix E and the network weights together.

Embeddings Support Generalization

Similar words can support similar predictions in contexts not observed exactly.

Embeddings Support Generalization

Similar words can support similar predictions in contexts not observed exactly.

OBSERVED DURING TRAINING

make sure that the **cat** gets → **fed**

Embeddings Support Generalization

Similar words can support similar predictions in contexts not observed exactly.

OBSERVED DURING TRAINING

make sure that the **cat** gets → **fed**

NEW CONTEXT

make sure that the **dog** gets → ?

Embeddings Support Generalization

Similar words can support similar predictions in contexts not observed exactly.

OBSERVED DURING TRAINING

make sure that the **cat** gets $\rightarrow$ **fed**

NEW CONTEXT

make sure that the **dog** gets $\rightarrow$?

$$E[\text{cat}] \approx E[\text{dog}] \implies \mathbf{h}_{\text{cat context}} \approx \mathbf{h}_{\text{dog context}}$$

This is a representational intuition, not a claim that the vectors must be identical.

Shared parameters let evidence from one context influence a related unseen context.

Embeddings Support Generalization

Similar words can support similar predictions in contexts not observed exactly.

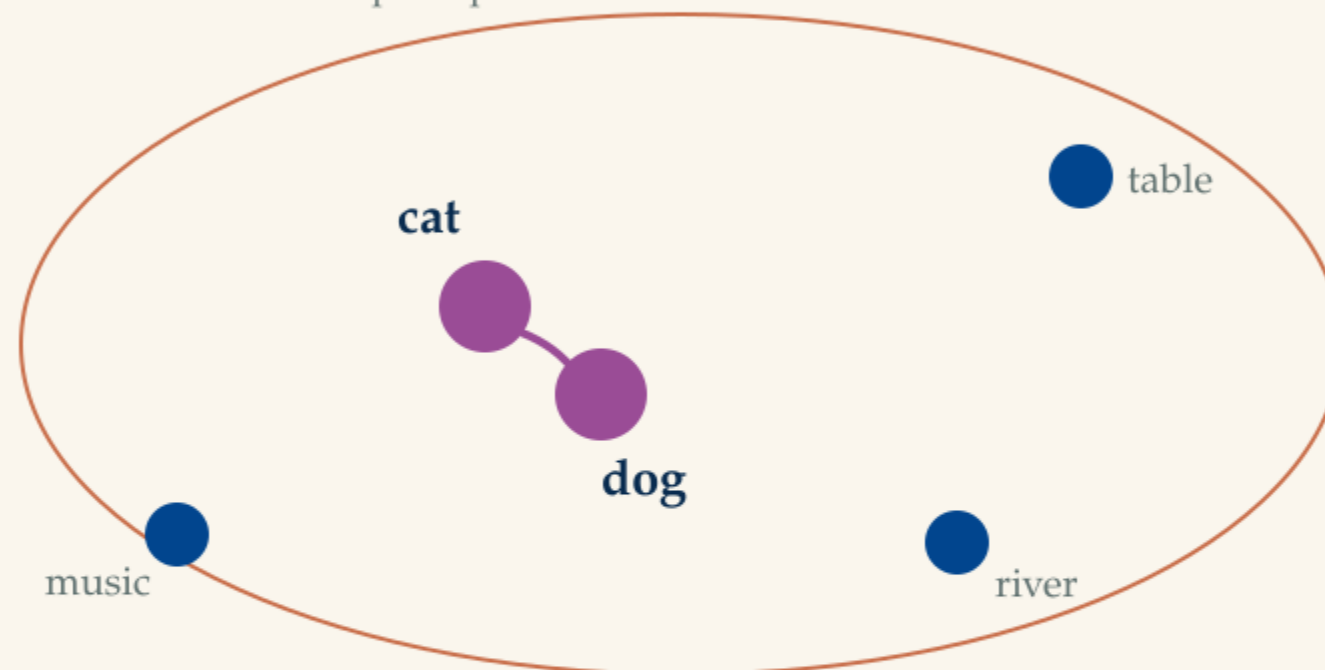
OBSERVED DURING TRAINING

make sure that the **cat** gets $\rightarrow$ **fed**

NEW CONTEXT

make sure that the **dog** gets $\rightarrow$?

SCHEMATIC EMBEDDING SPACE
Conceptual positions—not measured coordinates



$$E[\text{cat}] \approx E[\text{dog}]$$

$$\implies \mathbf{h}_{\text{cat context}} \approx \mathbf{h}_{\text{dog context}}$$

This is a representational intuition, not a claim that the vectors must be identical.

Shared parameters let evidence from one context influence a related unseen context.

The Model Still Sees a Fixed Context Window

Information before the chosen window is unavailable to the prediction.

EARLIER CONTEXT — OUTSIDE THE MODEL INPUT

THREE-TOKEN INPUT WINDOW

REQUIRED NEXT TOKEN

The **key** to the

old wooden cabinets

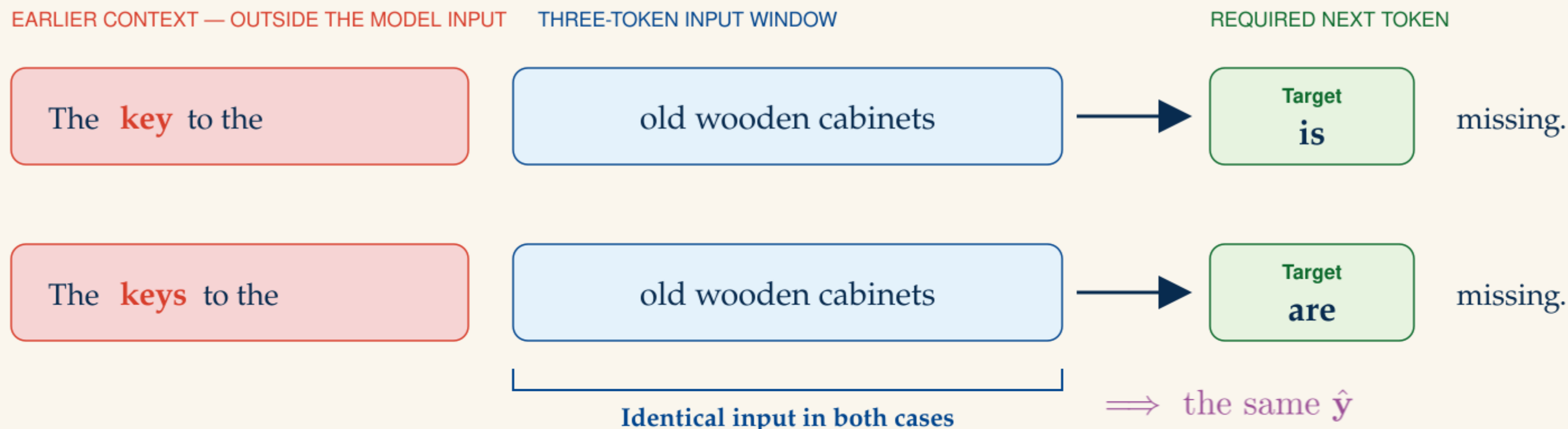


Target
is

missing.

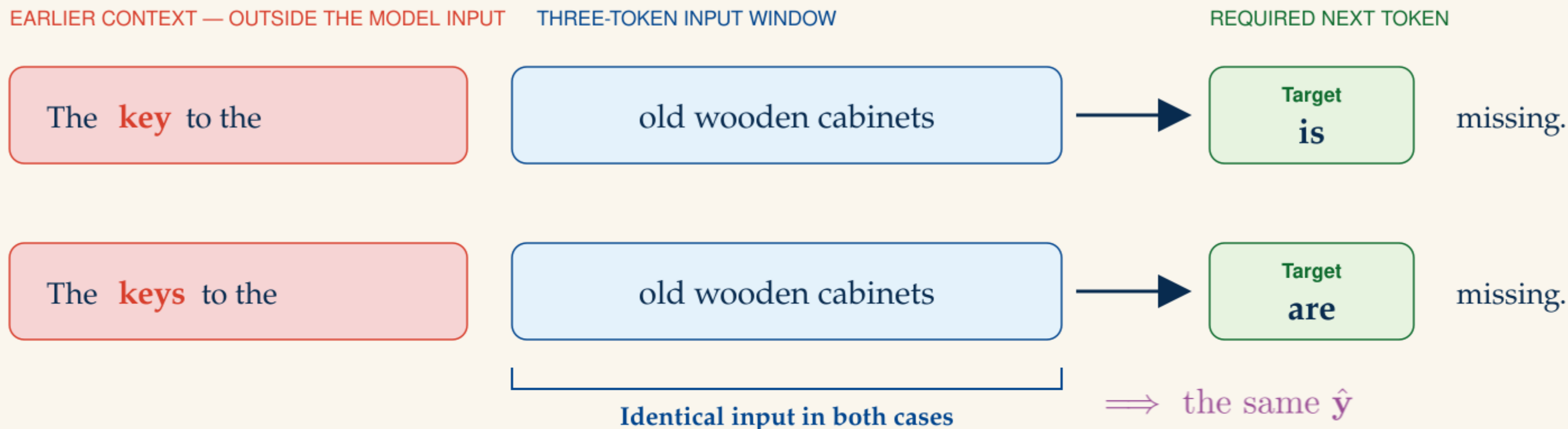
The Model Still Sees a Fixed Context Window

Information before the chosen window is unavailable to the prediction.



The Model Still Sees a Fixed Context Window

Information before the chosen window is unavailable to the prediction.



$$P(w_t \mid w_1, \dots, w_{t-1}) \approx P(w_t \mid w_{t-3}, w_{t-2}, w_{t-1})$$

The subject that determines agreement lies outside the window.

A larger window moves the boundary; it does not remove the boundary.

An RNN Processes an Ordered Sequence of Vectors

The recurrence can operate on sensor measurements, audio, video features, or text representations.

POSSIBLE SOURCES OF THE INPUT VECTOR

Sensor reading

One or more measurements at time t

$$x_t \in \mathbb{R}$$

Audio frame

Features from a short interval of sound

$$x_t \in \mathbb{R}^d$$

Video-frame features

Features extracted from the current frame

$$x_t \in \mathbb{R}^d$$

Word embedding

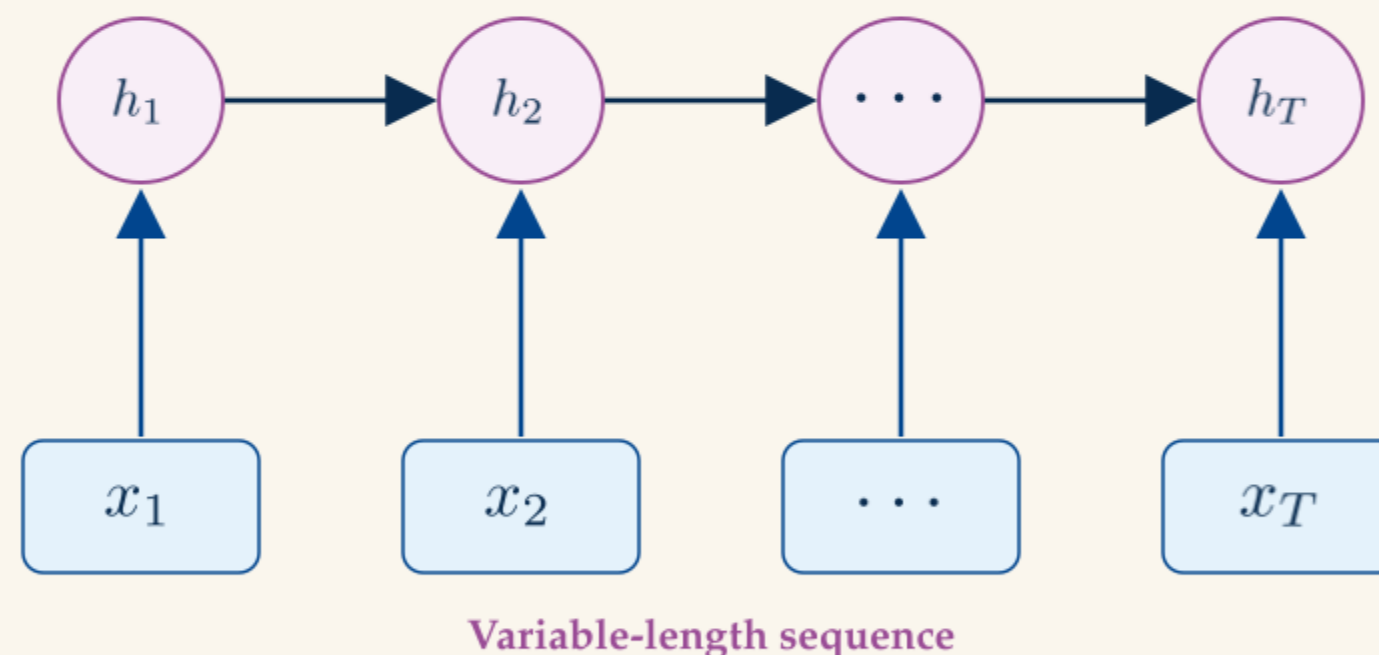
A vector representing the current token

$$e_t \in \mathbb{R}^d$$

THE SAME RECURRENT COMPUTATION

The RNN receives one vector at each step.

Only the vector values and their order enter the recurrence.



The model sees an ordered sequence of vectors—not the original modality itself.

The Hidden State Represents the Signal So Far

Each observation updates—rather than replaces—the representation of the past.

The Hidden State Represents the Signal So Far

Each observation updates—rather than replaces—the representation of the past.

A HAND-DESIGNED RECURRENCE FOR INTUITION

$$h_t = \frac{1}{2}h_{t-1} + x_t, \quad h_0 = 0$$

The new observation is combined with a retained part of the preceding state.

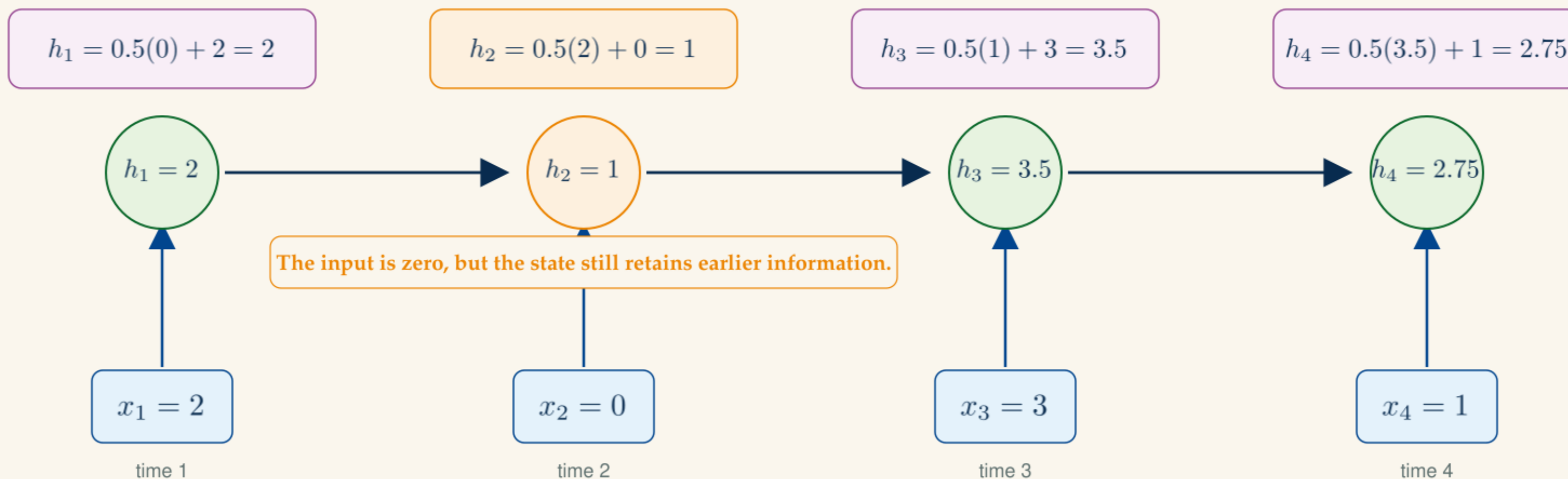
The Hidden State Represents the Signal So Far

Each observation updates—rather than replaces—the representation of the past.

A HAND-DESIGNED RECURRENCE FOR INTUITION

$$h_t = \frac{1}{2}h_{t-1} + x_t, \quad h_0 = 0$$

The new observation is combined with a retained part of the preceding state.



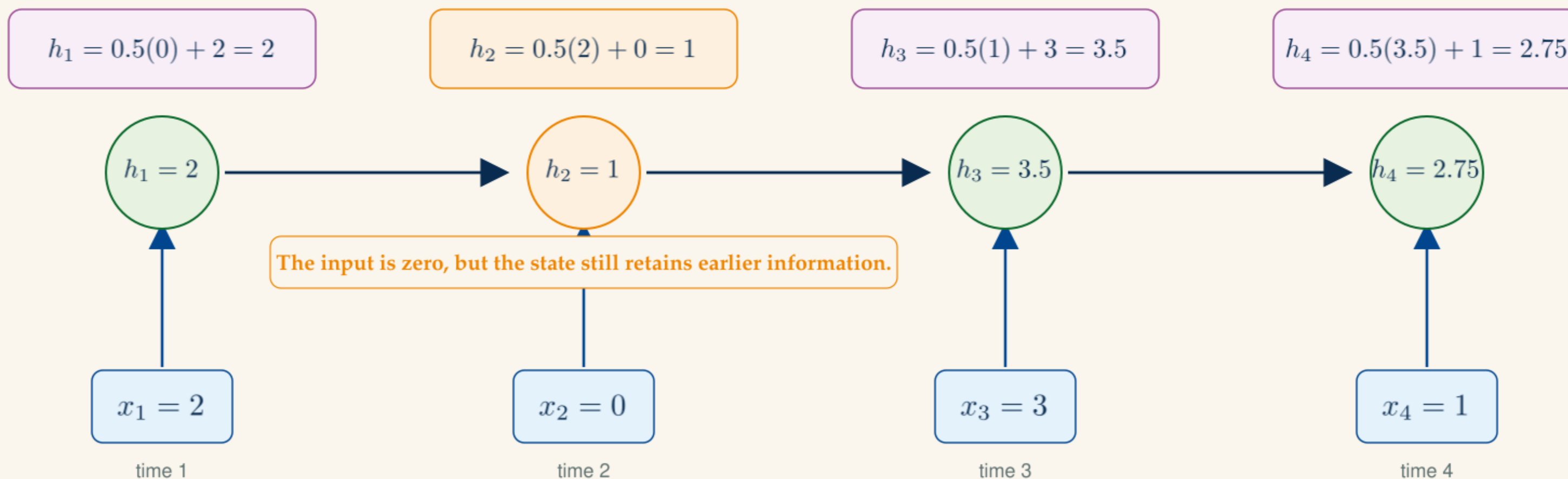
The Hidden State Represents the Signal So Far

Each observation updates—rather than replaces—the representation of the past.

A HAND-DESIGNED RECURRENCE FOR INTUITION

$$h_t = \frac{1}{2}h_{t-1} + x_t, \quad h_0 = 0$$

The new observation is combined with a retained part of the preceding state.



A vanilla RNN learns the update instead of fixing the coefficient by hand.

A Wearable Produces a Vector at Every Time Step

Three accelerometer channels become an ordered sequence of measurement vectors.

A Wearable Produces a Vector at Every Time Step

Three accelerometer channels become an ordered sequence of measurement vectors.

BODY-WORN SMARTPHONE ACCELEROMETER

Sitting

Transition

Standing

Walking

ONE COLUMN AT TIME t

The sensor reports three numbers:

A Wearable Produces a Vector at Every Time Step

Three accelerometer channels become an ordered sequence of measurement vectors.

BODY-WORN SMARTPHONE ACCELEROMETER

Sitting

Transition

Standing

Walking

ONE COLUMN AT TIME t

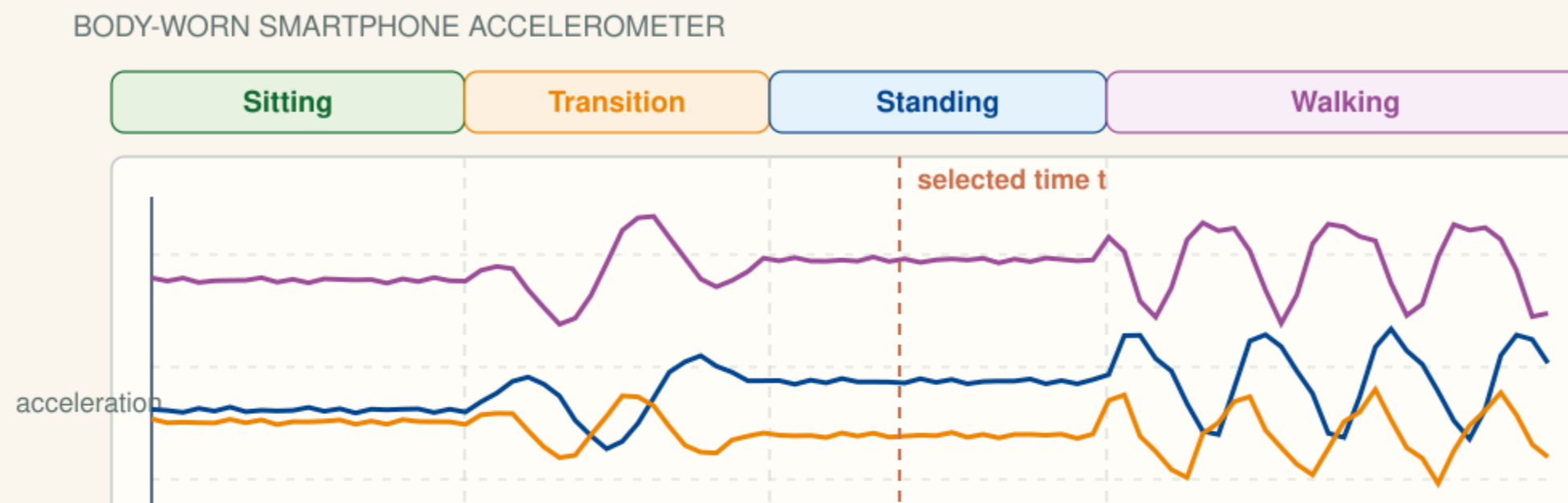
The sensor reports three numbers:

$$\mathbf{x}_t = (a_x(t), a_y(t), a_z(t))^T \in \mathbb{R}^3$$

one measurement vector

A Wearable Produces a Vector at Every Time Step

Three accelerometer channels become an ordered sequence of measurement vectors.



ONE COLUMN AT TIME t

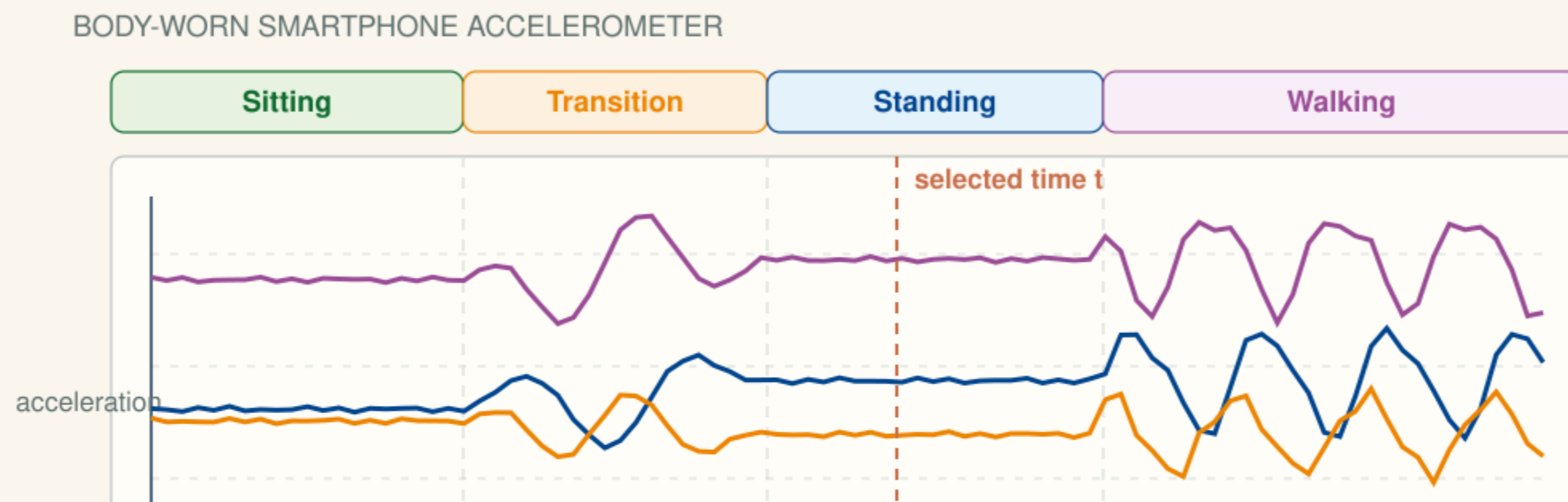
The sensor reports three numbers:

$$\mathbf{x}_t = (a_x(t), a_y(t), a_z(t))^T \in \mathbb{R}^3$$

one measurement vector

A Wearable Produces a Vector at Every Time Step

Three accelerometer channels become an ordered sequence of measurement vectors.



ONE COLUMN AT TIME t

The sensor reports three numbers:

$$\mathbf{x}_t = (a_x(t), a_y(t), a_z(t))^T \in \mathbb{R}^3$$

one measurement vector

The complete recording is a matrix:

$$X = [\mathbf{x}_1 \ \mathbf{x}_2 \ \cdots \ \mathbf{x}_T] \in \mathbb{R}^{3 \times T}$$

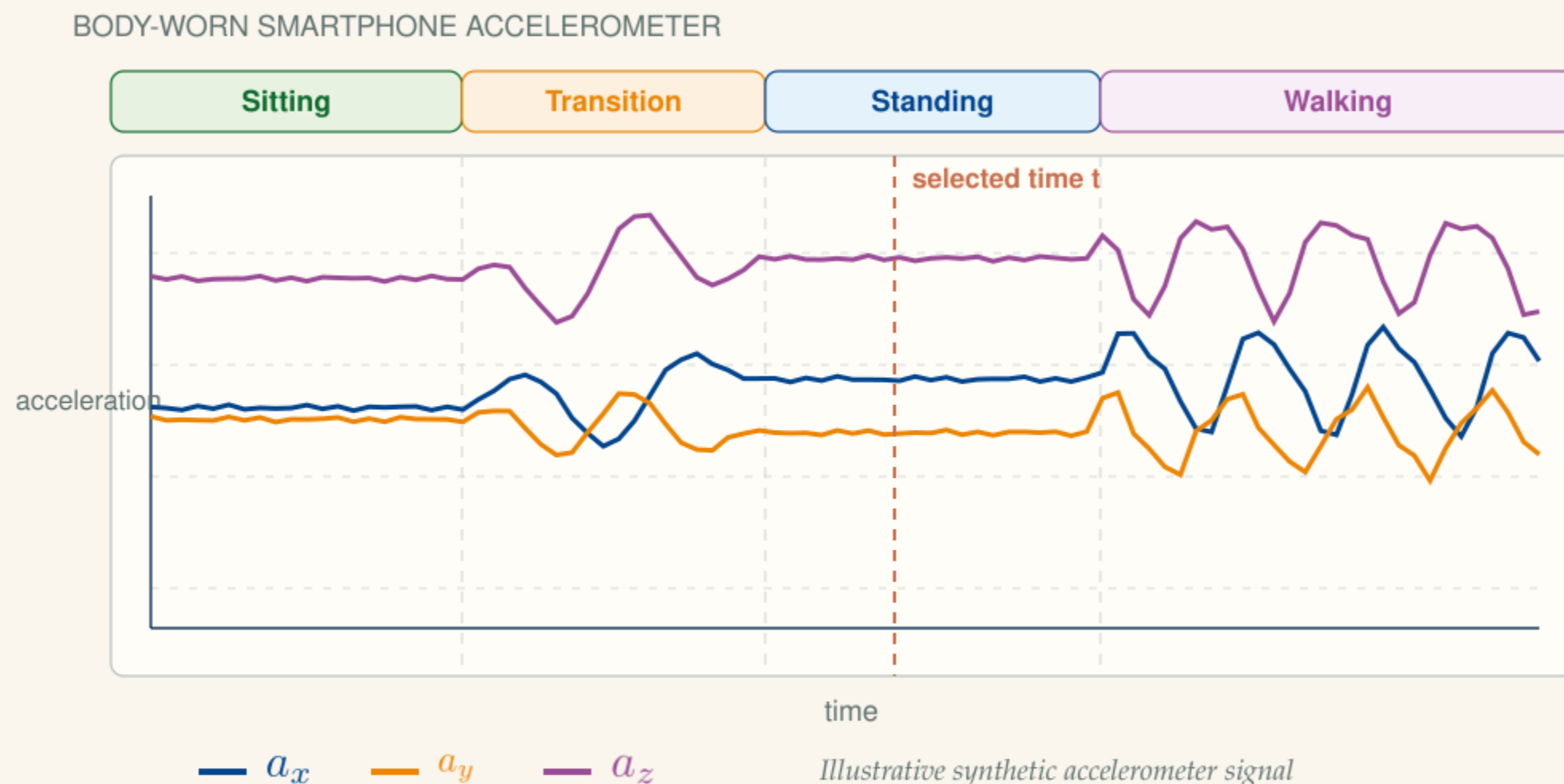
columns are ordered in time

Possible labels at every time step:

Sitting · Standing · Walking · Running · Dancing

A Wearable Produces a Vector at Every Time Step

Three accelerometer channels become an ordered sequence of measurement vectors.



ONE COLUMN AT TIME t

The sensor reports three numbers:

$$\mathbf{x}_t = (a_x(t), a_y(t), a_z(t))^T \in \mathbb{R}^3$$

one measurement vector

The complete recording is a matrix:

$$X = [\mathbf{x}_1 \ \mathbf{x}_2 \ \cdots \ \mathbf{x}_T] \in \mathbb{R}^{3 \times T}$$

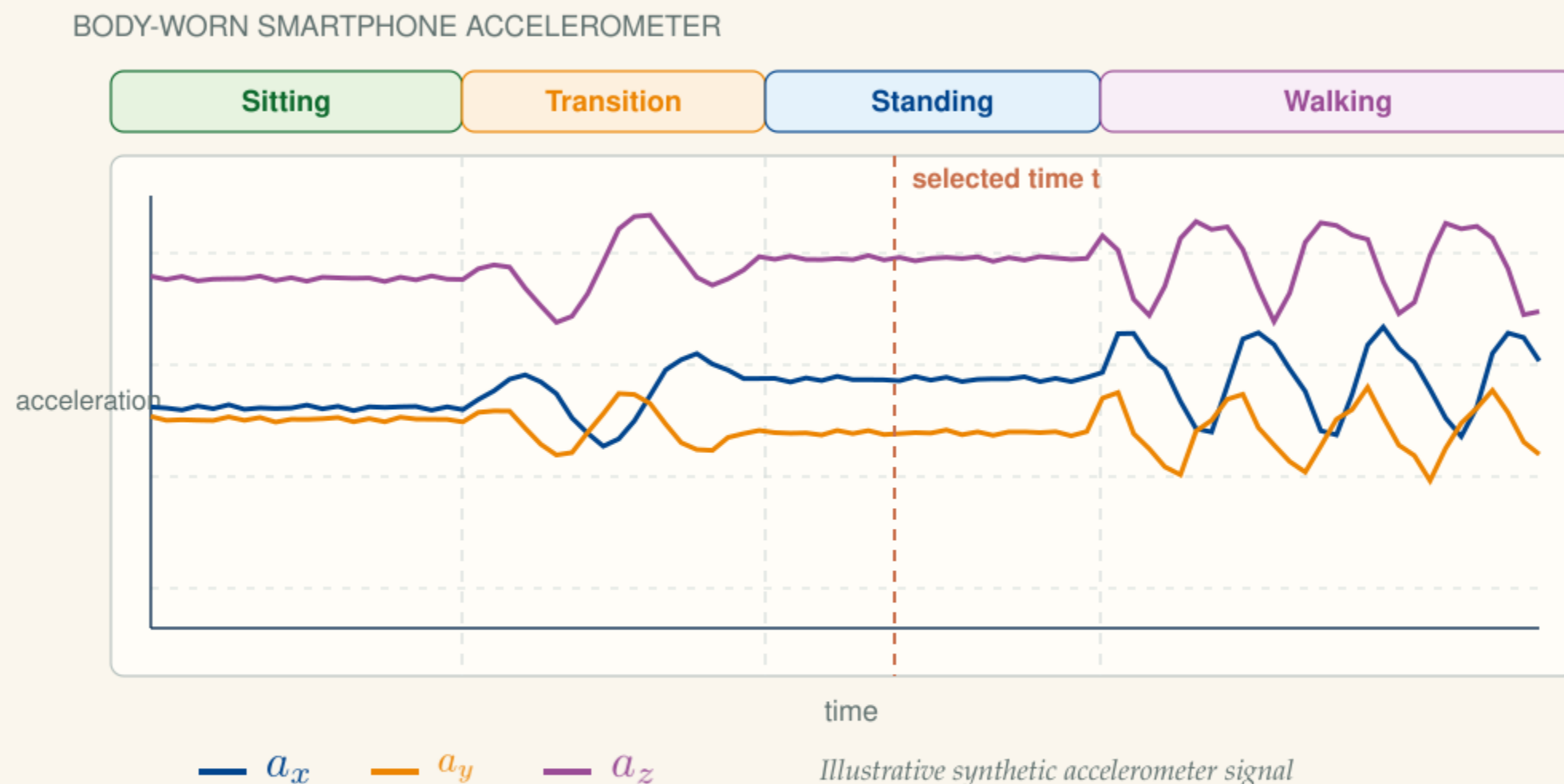
columns are ordered in time

Possible labels at every time step:

Sitting · Standing · Walking · Running · Dancing

A Wearable Produces a Vector at Every Time Step

Three accelerometer channels become an ordered sequence of measurement vectors.



ONE COLUMN AT TIME t

The sensor reports three numbers:

$$\mathbf{x}_t = (a_x(t), a_y(t), a_z(t))^T \in \mathbb{R}^3$$

one measurement vector

The complete recording is a matrix:

$$X = [\mathbf{x}_1 \ \mathbf{x}_2 \ \cdots \ \mathbf{x}_T] \in \mathbb{R}^{3 \times T}$$

columns are ordered in time

Possible labels at every time step:

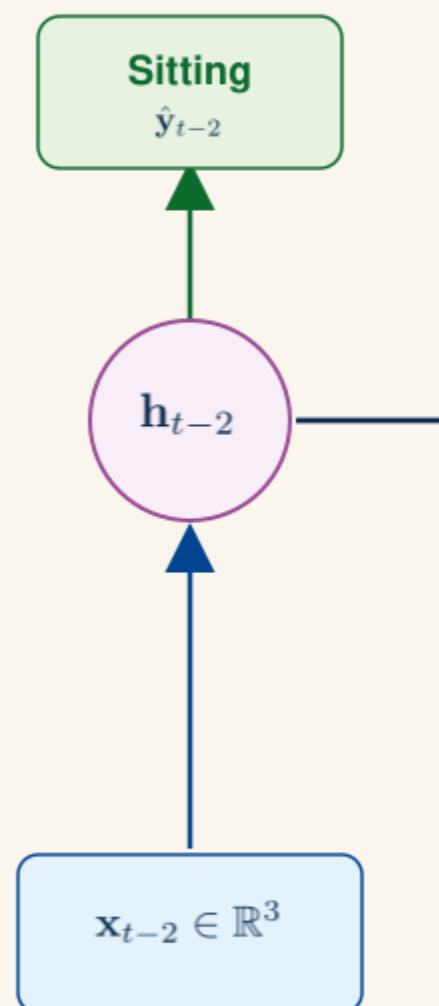
Sitting · Standing · Walking · Running · Dancing

Each vertical slice of the three-channel signal is one input vector.

Motion History Becomes an Activity Label

A recurrent state turns evolving sensor patterns into decisions at each time step.

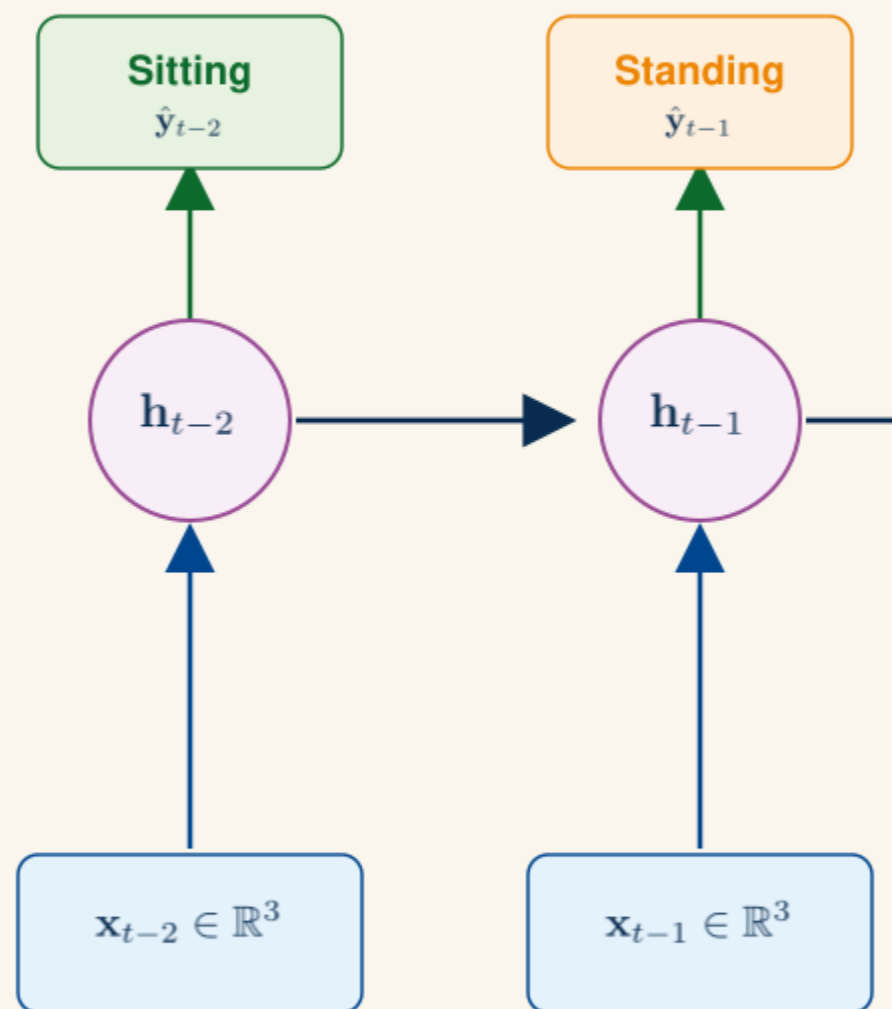
THE SAME RECURRENCE RUNS AS MEASUREMENTS ARRIVE



Motion History Becomes an Activity Label

A recurrent state turns evolving sensor patterns into decisions at each time step.

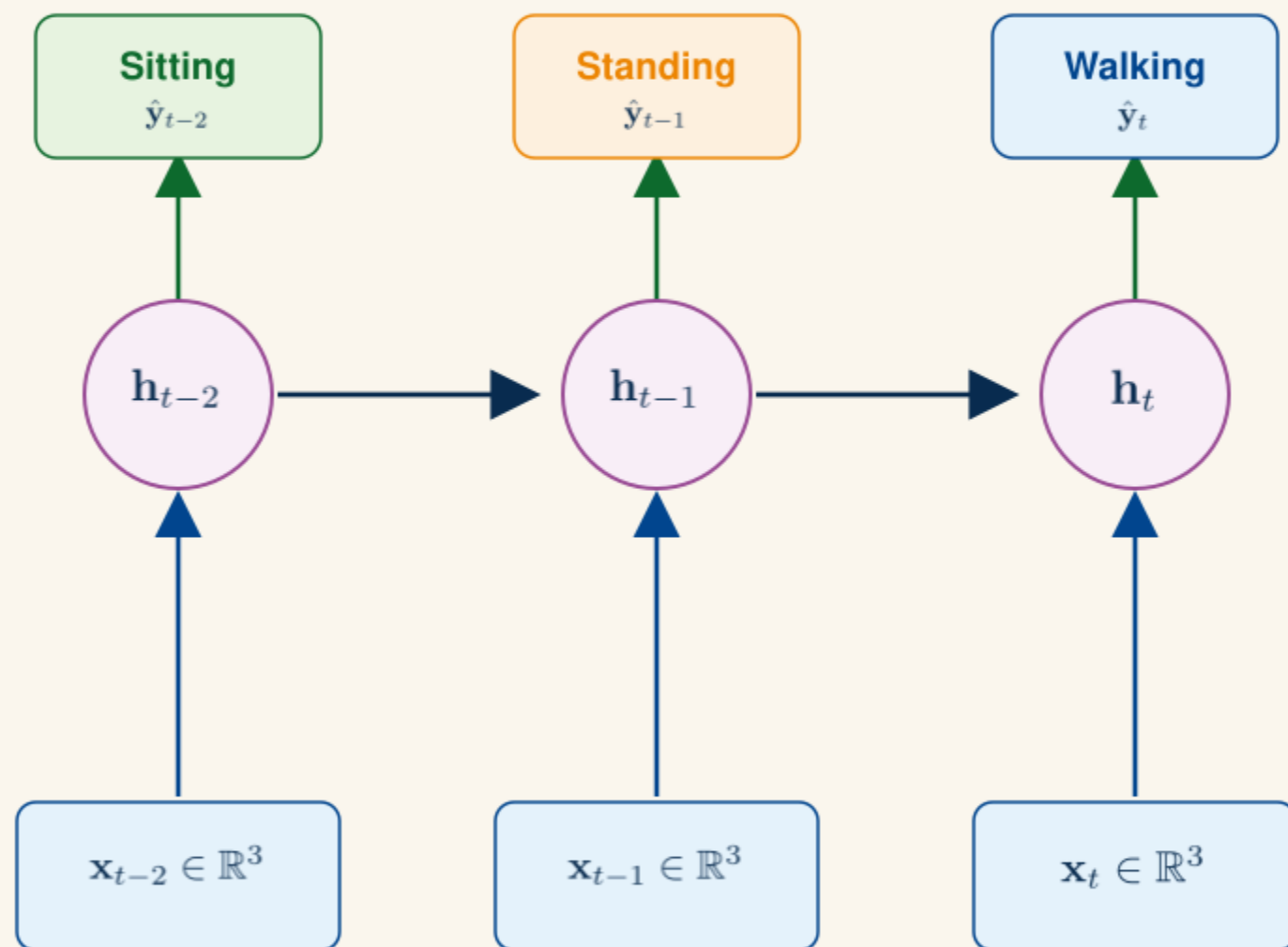
THE SAME RECURRENCE RUNS AS MEASUREMENTS ARRIVE



Motion History Becomes an Activity Label

A recurrent state turns evolving sensor patterns into decisions at each time step.

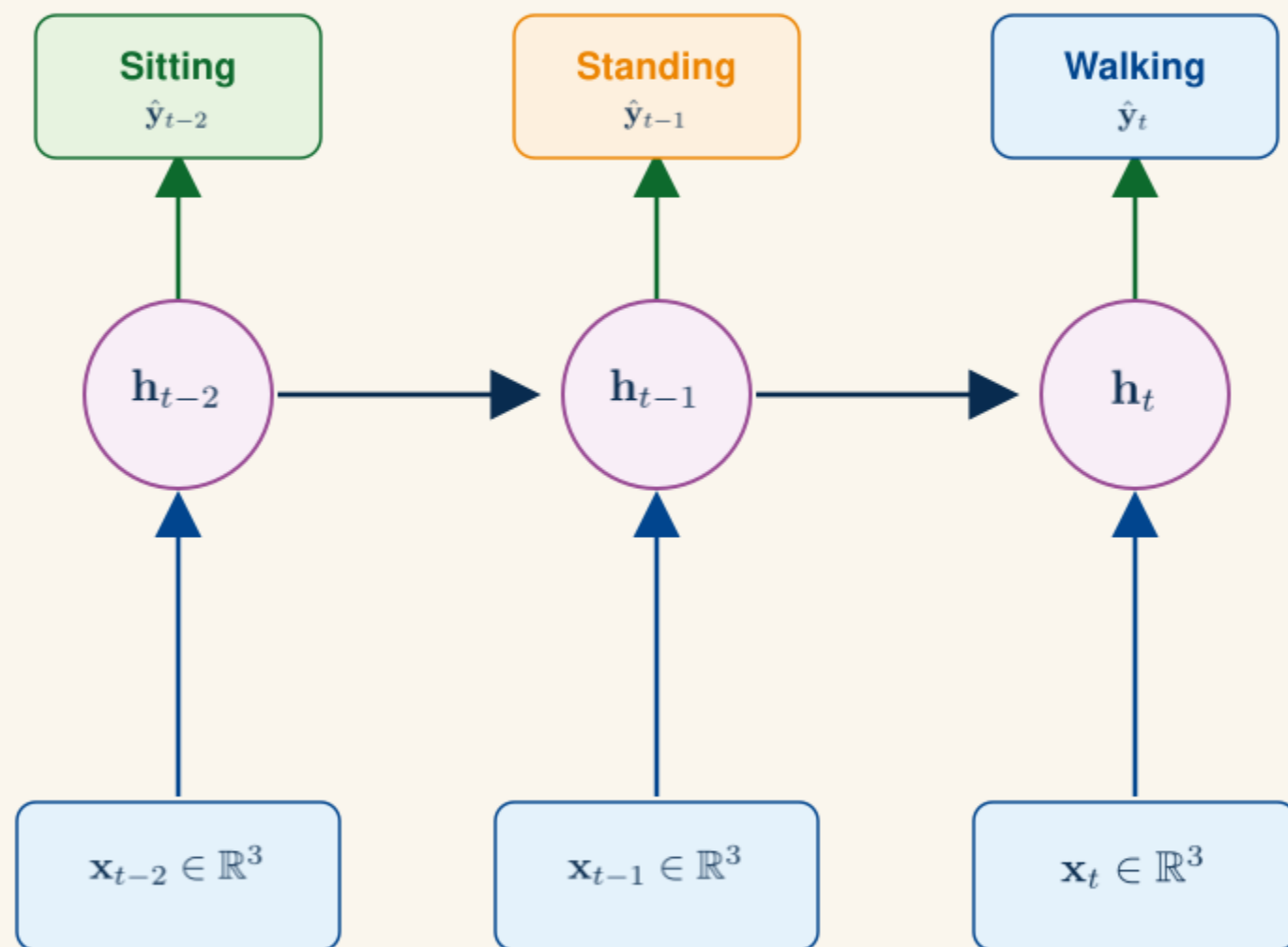
THE SAME RECURRENCE RUNS AS MEASUREMENTS ARRIVE



Motion History Becomes an Activity Label

A recurrent state turns evolving sensor patterns into decisions at each time step.

THE SAME RECURRENCE RUNS AS MEASUREMENTS ARRIVE



VECTOR DIMENSIONS IN THIS EXAMPLE

Update the running state

$$\mathbf{h}_t = g(U\mathbf{h}_{t-1} + W\mathbf{x}_t + \mathbf{b}_h)$$

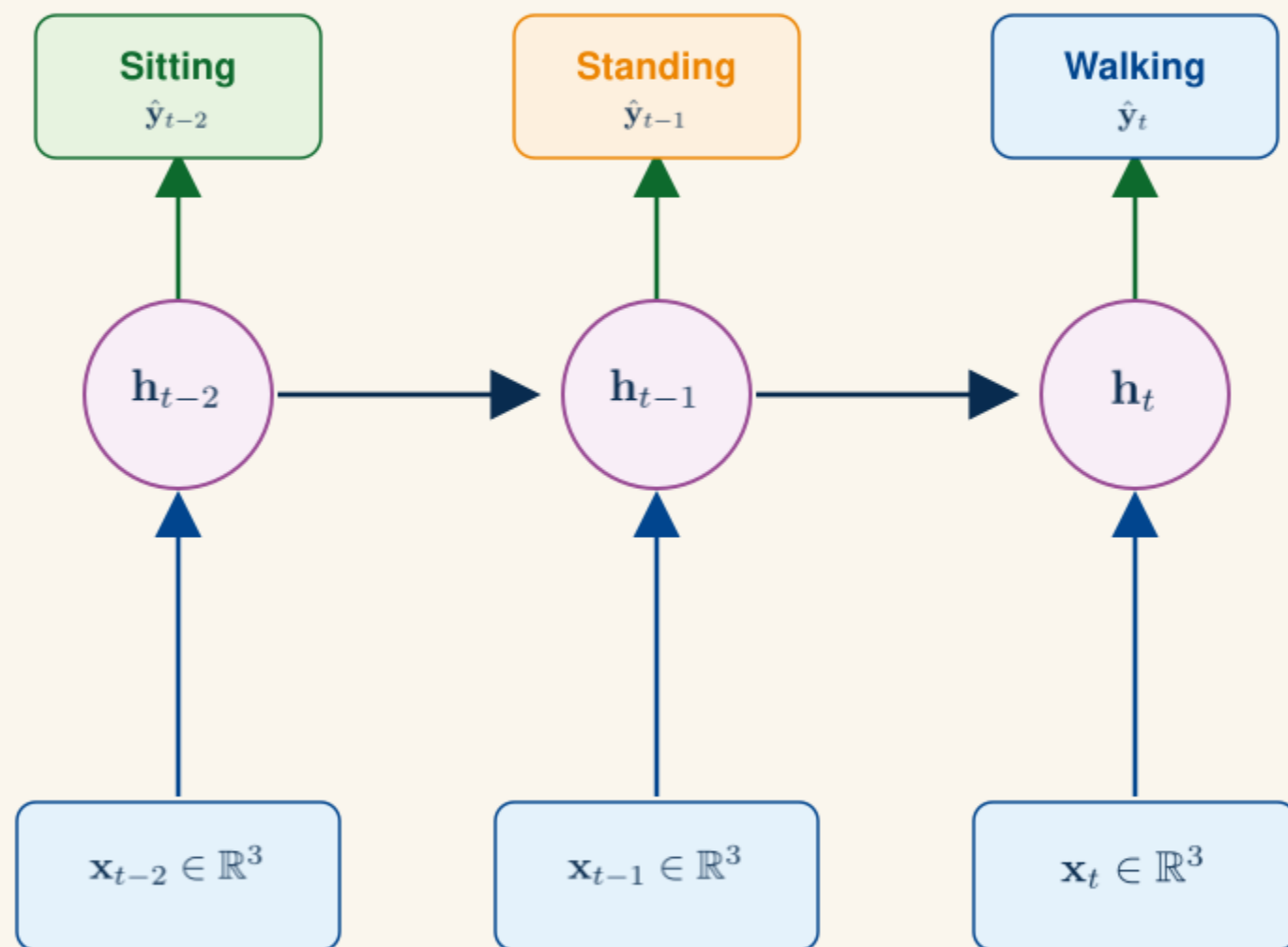
Current evidence $W\mathbf{x}_t \in \mathbb{R}^m$

Carried history $U\mathbf{h}_{t-1} \in \mathbb{R}^m$

Motion History Becomes an Activity Label

A recurrent state turns evolving sensor patterns into decisions at each time step.

THE SAME RECURRENCE RUNS AS MEASUREMENTS ARRIVE



VECTOR DIMENSIONS IN THIS EXAMPLE

Update the running state

$$\mathbf{h}_t = g(U\mathbf{h}_{t-1} + W\mathbf{x}_t + \mathbf{b}_h)$$

Current evidence $W\mathbf{x}_t \in \mathbb{R}^m$

Carried history $U\mathbf{h}_{t-1} \in \mathbb{R}^m$

Predict the activity at time t

$$\hat{\mathbf{y}}_t = \text{softmax}(V\mathbf{h}_t + \mathbf{b}_y)$$

Input $\mathbf{x}_t \in \mathbb{R}^3$ State $\mathbf{h}_t \in \mathbb{R}^m$

Output $\hat{\mathbf{y}}_t \in \mathbb{R}^5$

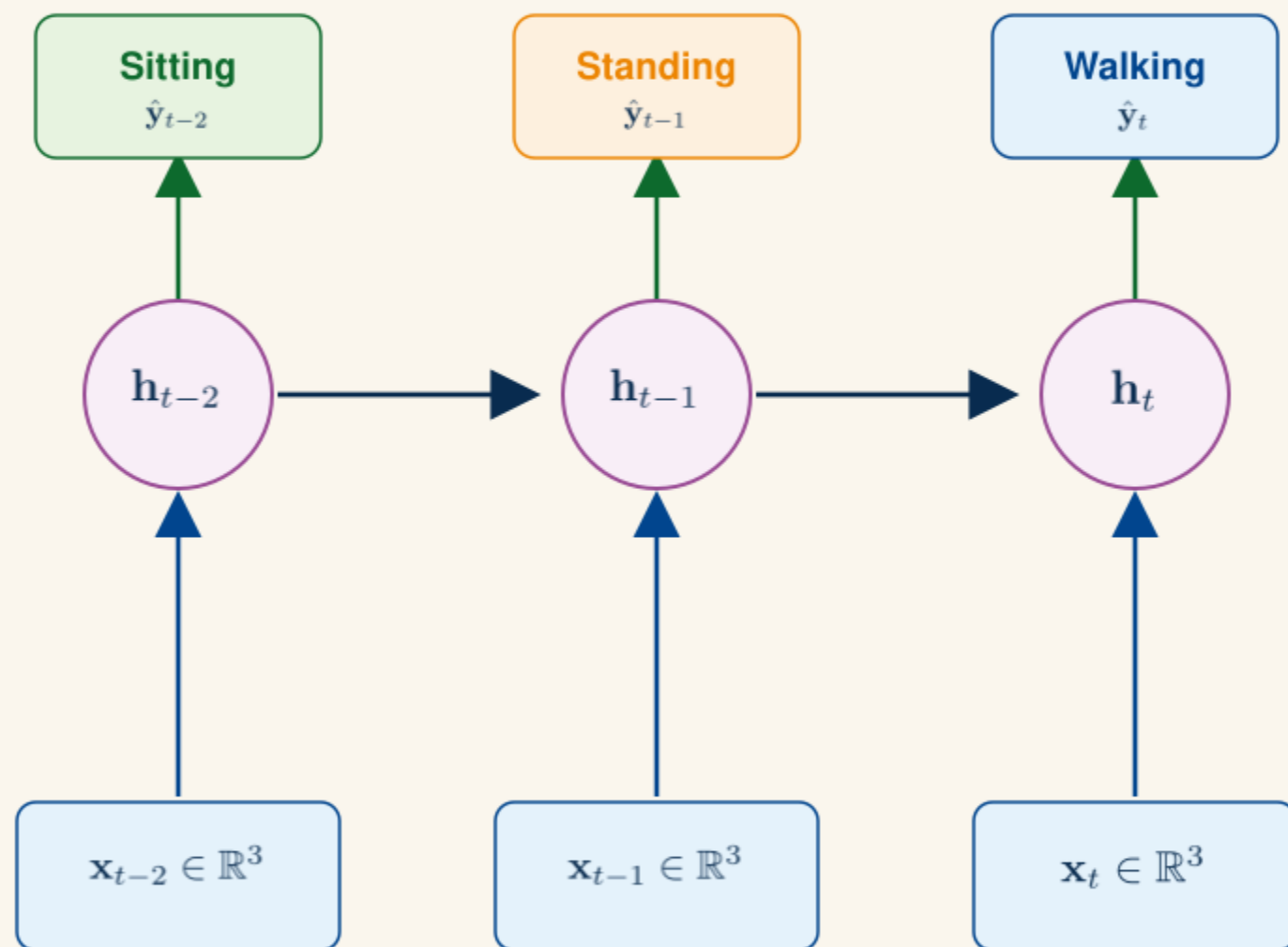
Why history helps

transitions and repeated gait patterns unfold across time

Motion History Becomes an Activity Label

A recurrent state turns evolving sensor patterns into decisions at each time step.

THE SAME RECURRENCE RUNS AS MEASUREMENTS ARRIVE



VECTOR DIMENSIONS IN THIS EXAMPLE

Update the running state

$$\mathbf{h}_t = g(U\mathbf{h}_{t-1} + W\mathbf{x}_t + \mathbf{b}_h)$$

Current evidence $W\mathbf{x}_t \in \mathbb{R}^m$

Carried history $U\mathbf{h}_{t-1} \in \mathbb{R}^m$

Predict the activity at time t

$$\hat{\mathbf{y}}_t = \text{softmax}(V\mathbf{h}_t + \mathbf{b}_y)$$

Input $\mathbf{x}_t \in \mathbb{R}^3$ State $\mathbf{h}_t \in \mathbb{R}^m$

Output $\hat{\mathbf{y}}_t \in \mathbb{R}^5$

Why history helps

transitions and repeated gait patterns unfold across time

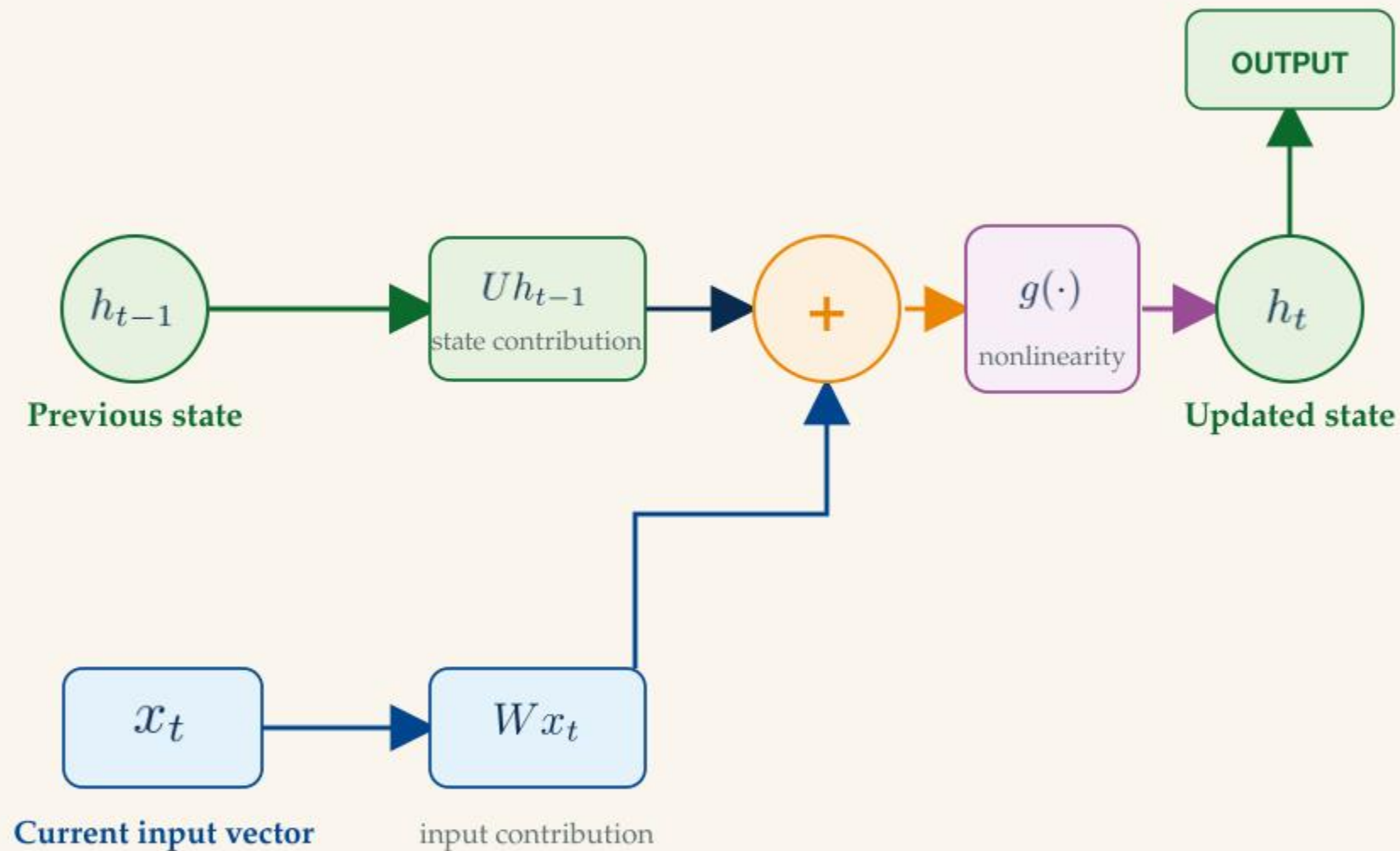
A Vanilla RNN Learns How to Update Its State

The same learned transformation is applied as each new input vector arrives.

A Vanilla RNN Learns How to Update Its State

The same learned transformation is applied as each new input vector arrives.

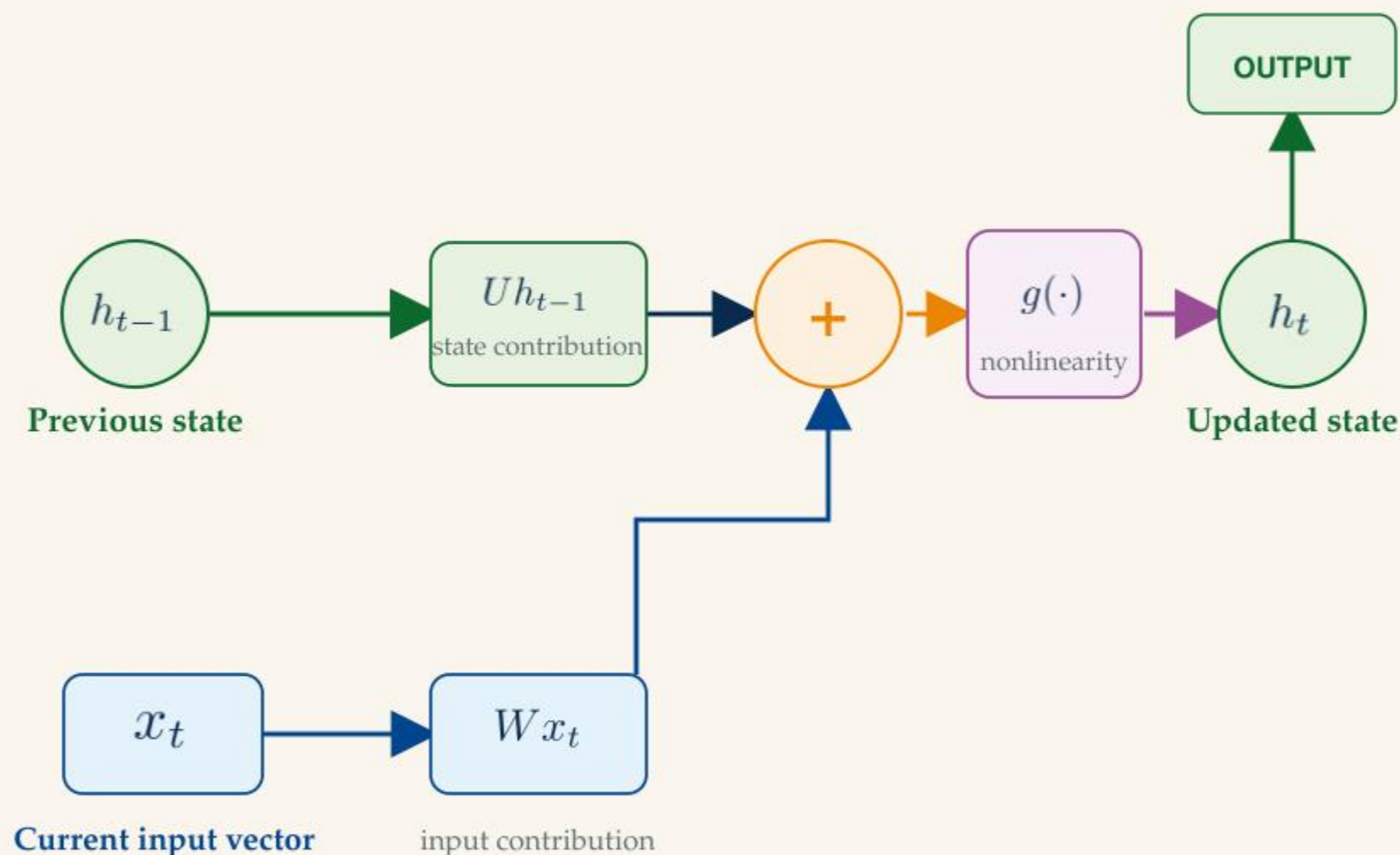
ONE VANILLA-RNN STEP



A Vanilla RNN Learns How to Update Its State

The same learned transformation is applied as each new input vector arrives.

ONE VANILLA-RNN STEP



THE LEARNED UPDATE

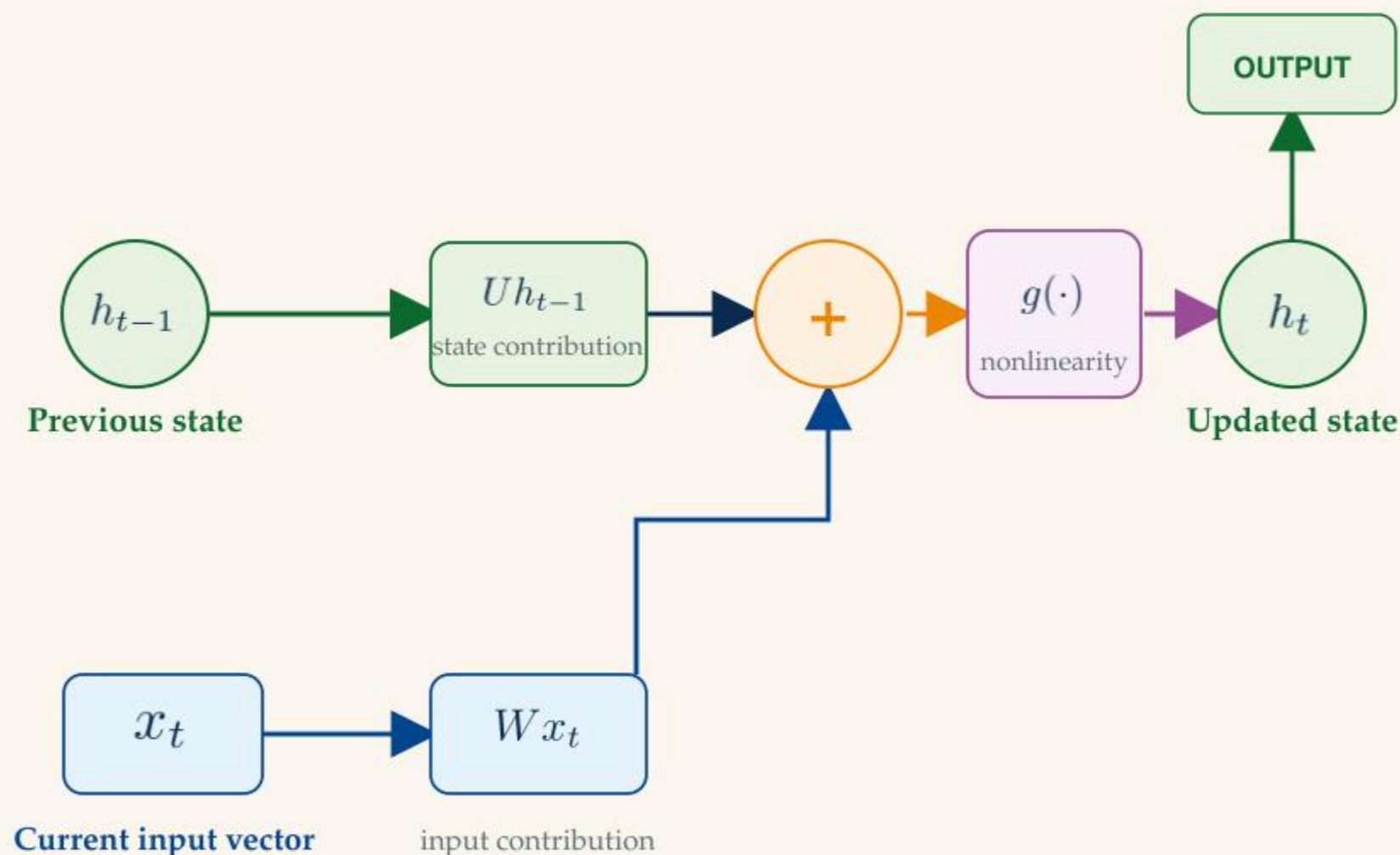
Update the hidden state

$$h_t = g(Uh_{t-1} + Wx_t + b_h)$$

A Vanilla RNN Learns How to Update Its State

The same learned transformation is applied as each new input vector arrives.

ONE VANILLA-RNN STEP



THE LEARNED UPDATE

Update the hidden state

$$h_t = g(Uh_{t-1} + Wx_t + b_h)$$

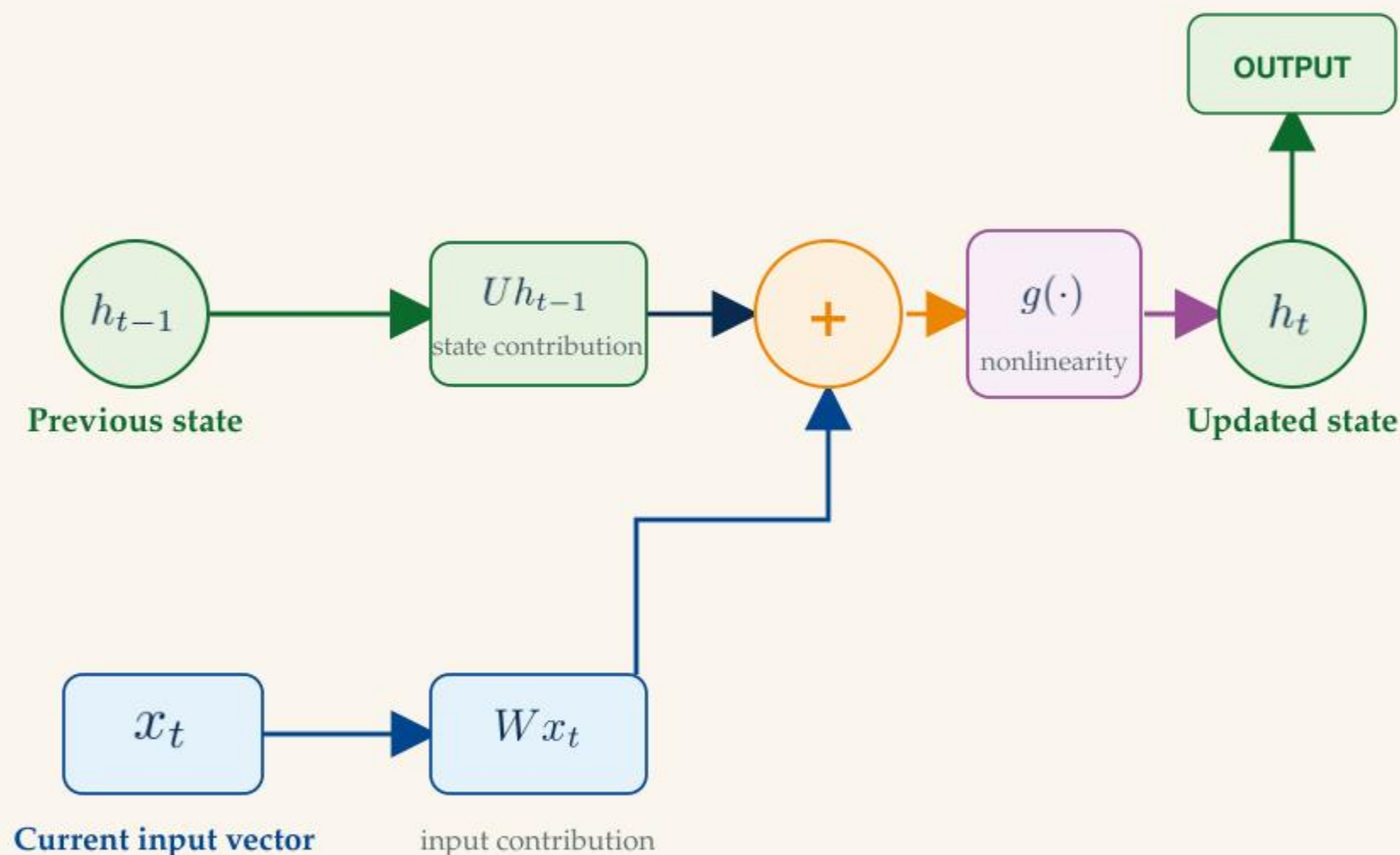
Produce a task-specific output

$$\hat{y}_t = f(Vh_t + b_y)$$

A Vanilla RNN Learns How to Update Its State

The same learned transformation is applied as each new input vector arrives.

ONE VANILLA-RNN STEP



THE LEARNED UPDATE

Update the hidden state

$$h_t = g(Uh_{t-1} + Wx_t + b_h)$$

Produce a task-specific output

$$\hat{y}_t = f(Vh_t + b_y)$$

Parameter roles

$$U : h \rightarrow h \quad W : x \rightarrow h \quad V : h \rightarrow y$$

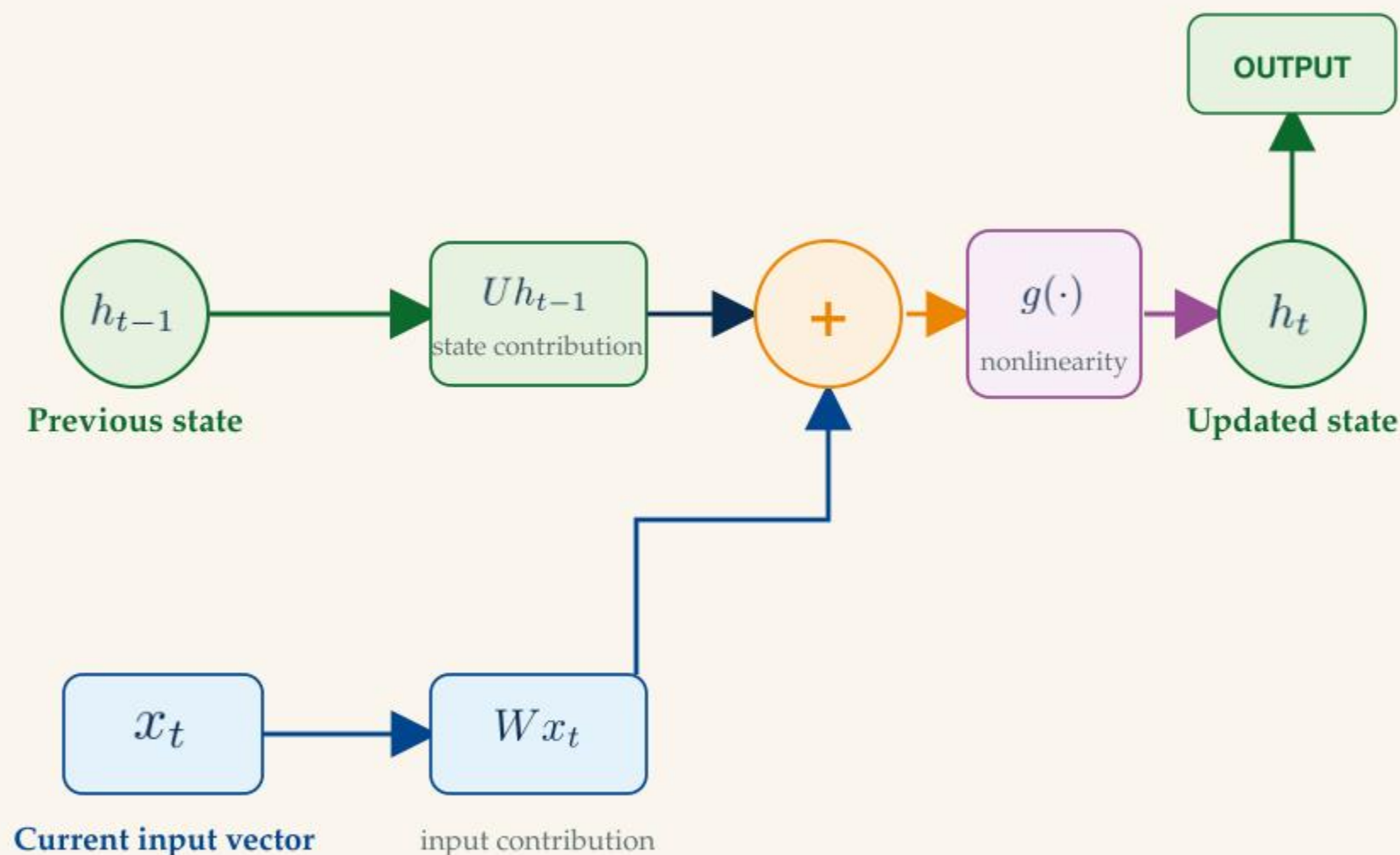
The matrices are learned and reused at every time step.

Elman (1990): the simple recurrent network.

A Vanilla RNN Learns How to Update Its State

The same learned transformation is applied as each new input vector arrives.

ONE VANILLA-RNN STEP



THE LEARNED UPDATE

Update the hidden state

$$h_t = g(Uh_{t-1} + Wx_t + b_h)$$

Produce a task-specific output

$$\hat{y}_t = f(Vh_t + b_y)$$

Parameter roles

$$U : h \rightarrow h \quad W : x \rightarrow h \quad V : h \rightarrow y$$

The matrices are learned and reused at every time step.

Elman (1990): the simple recurrent network.

For text, the input vector is usually a learned word or token embedding.